Contents

1 Introduction **3**

2 Rocket 2D **4**
2.1 Frames . . . 4
2.2 Forces and moments . . . 4
2.3 Geometry . . . 4
2.4 Equations of motion . . . 5

3 Models **6**
3.1 Gravity . . . 6
3.1.1 Magnitude . . . 6
3.1.2 Direction . . . 6
3.1.3 Vector field (magnitude + direction) . . . 7
3.1.4 Position relative to O and true altitude . . . 7
3.2 Thrust . . . 8
3.2.1 Magnitude . . . 8
3.2.2 Direction . . . 8
3.3 Mass, centre of gravity and moment of inertia . . . 8
3.3.1 Mass . . . 8
3.3.2 Centre of gravity . . . 9
3.3.3 Moment of inertia . . . 9
3.3.4 Propellant grain geometry . . . 10
3.4 Aerodynamics . . . 10
3.4.1 Relative velocity and angle of attack . . . 10
3.4.2 Force and moment . . . 10
3.4.3 Dynamic pressure . . . 11
3.4.4 Coefficients . . . 11
3.5 Atmosphere . . . 12
3.5.1 Model tiers . . . 12
3.5.2 Density . . . 13
3.5.3 Reynolds and Mach numbers . . . 14
3.6 Wind . . . 14
3.6.1 Vertical wind . . . 15
3.7 Noise and disturbances . . . 15
3.7.1 Thrust noise . . . 15
3.7.2 Atmospheric disturbances . . . 15
3.8 Non-inertial frame corrections (Earth's rotation) . . . 16
3.9 States, initial conditions and integration . . . 18
3.10 Sampled-data evaluation (zero-order hold) . . . 19

4 Other dynamics models **19**
4.1 Launch rail . . . 20
4.2 Recovery . . . 20
4.3 Air brake . . . 21
4.3.1 Deployment law . . . 22
4.3.2 Deployment trigger . . . 22
4.4 Roll and the reaction wheel . . . 23
4.4.1 Why roll decouples exactly . . . 23
4.4.2 Equations of motion . . . 23
4.4.3 Authority and saturation . . . 24
4.4.4 Aerodynamic roll moment . . . 24
4.4.5 Control law and the constrained phases . . . 24

5 Rocket 3D **25**
5.1 What 3D changes, and what it does not . . . 25
5.2 Attitude: the quaternion . . . 25
5.3 The inertia tensor . . . 26
5.4 Which tensors are physically possible . . . 27
5.5 Rotational equations of motion . . . 27

5.6 The moment vector . . . 28
5.7 State vector, and what remains . . . 29

6 Analytical analysis **30**
6.1 Forces . . . 30
6.2 Moment . . . 30
6.2.1 No thrust: coast and descent . . . 30
6.2.2 With thrust, aligned . . . 30
6.2.3 With thrust, misaligned . . . 31
6.3 Static stability . . . 31
6.4 Flight-path angle and the point-mass gravity turn . . . 32
6.4.1 Dynamic stability . . . 32

7 Current assumptions **34**

8 Backlog and future work **35**

A Notation **37**

B Model inventory **39**

C Conventions **40**

D Constants **40**

1 Introduction

This report documents a simulator of rocket flight in the vertical plane, with room left for a later 3D extension, and the physics behind it. The simulator integrates a rigid rocket from the launch rail to landing, so that design choices — motor, mass, aerodynamics, recovery — can be compared by their effect on the trajectory. The report derives the equations the simulator solves and records the models and assumptions behind each term.

Around the simulation core (`apps/sim/engine/`) sit the inputs and the tools that use it:

- **Configurations and data.** Rocket, launch site, weather, motor and scenario descriptions (`apps/sim/schema.py`), with motor curves, aerodynamic tables and atmospheric soundings under `projects/data/`. The simulator itself is offline and deterministic; a sounding can be fetched separately with `apps/input/sounding_fetch/fetch_soundi`
- **Output applications:**
  - single simulation (2D or 3D);
  - motor sweep;
  - CFD input conditions (output);
  - mass–aerodynamics trade-off (weight and so on; can live under the same CFD tab);
  - and, more specific, air-brake sizing.

Planned sub-applications, each managing one group of inputs; configurations keep only data, and the interface presents the models:

- **Rocket manager.**
  - *Rocket constructor*, similar to OpenRocket: produces the dry-rocket parameters (m_r, $l_{g,r}$, $I_{r,g}$, ...). They can be overwritten by CAD or measured dry values, or entered directly without the constructor.
  - *Rocket aerodynamics*: choose the model (constant, linearized, lookup table) and the input variables of the coefficients (M, Re, α, ...).
  - *Wing-body aerodynamics approximation*: approximate the rocket's aerodynamics by a component build-up, with the drag of each component taken from research or semi-empirical data; components may be neglected.
- **Motor manager.**
  - Thrust-data import, and whether the pressure-correction model is applied.
  - Motor mass retriever: all specifications, as for the dry rocket, with graphs of their change over time.
- **Recovery manager.**
  - Parachute design for different shapes and sizes, with aerodynamic data.
  - Structural data, e.g. the force per shroud line.
  - Optionally, a spring–damper model of the riser.
- **Weather manager.** Choose, analyse and manage weather data: atmosphere and wind.
- **Site designer.** Launch-site altitude above the Earth's centre, optionally a ground-height model, and above all the rail.

Text in blue is unreviewed. Two kinds of to-do mark open work: (sim: implemented in the report, not yet in the simulator) and (research: needs research before it can be written down). Framed boxes marked "Figure (planned)" describe figures yet to be made, and those marked "Comparison (planned)" the model comparisons to be generated by the simulator on the demo rocket.

The report is organised as follows. section 2 sets up the frames, forces and equations of motion; section 3 gives the interchangeable models behind each term; section 6 works the equations through by hand and derives the stability results. section 7 lists the simplifications in force and section 8 what is left undone. The appendix holds the notation, the model inventory and the sign conventions.

2 Rocket 2D

This section sets up the rigid rocket in the pitch plane: its frames, the forces and moments and where they act, and the equations of motion. The models behind each term are in section 3.

2.1 Frames

Two frames are used.

- $\mathcal{F}_0$ — ground (launch) frame, origin at the launch site O^*, with $\hat{x}_0$ horizontal downrange and $\hat{z}_0$ down.
- $\mathcal{F}_r$ — rocket (body) frame, with $\hat{x}_r$ along the nose axis, forward.

The state is integrated in $\mathcal{F}_0$: position $\vec{x}_r = x\,\hat{x}_0 + z\,\hat{z}_0$, velocity $\vec{v}_r$ and acceleration $\vec{a}_r$. Models that need the position relative to the Earth's centre O use $\vec{Og} = \vec{OO^*} + \vec{O^*g}$, defined where they are introduced.

In $\mathcal{F}_r$, $\hat{z}_r$ completes the pitch plane and $\hat{y}_r$ points out of it. The pitch angle θ_r is measured from $\hat{x}_0$, positive nose-up; ω_r and ε_r are the pitch rate and pitch acceleration. In ground-frame components,

$$\hat{x}_r = \cos\theta_r\,\hat{x}_0 - \sin\theta_r\,\hat{z}_0, \qquad \hat{z}_r = \sin\theta_r\,\hat{x}_0 + \cos\theta_r\,\hat{z}_0. \tag{2.1.1}$$

The gravity direction $\hat{z}_g$ (subsection 3.1) and the line of thrust $\hat{x}_t$ (Equation 3.2.4) are defined with the models that own them. All sign and axis conventions are collected in Appendix C.

2.2 Forces and moments

Weight is the main force the rocket has to overcome to gain altitude:

$$\vec{W} = mg\hat{z}_g, \tag{2.2.1}$$

with m the rocket's mass and g the gravitational acceleration along the local vertical $\hat{z}_g$; both are supplied by subsection 3.1.

Thrust comes from ejecting propellant at high velocity, plus the pressure difference across the nozzle exit. With the thrust direction of Equation 3.2.4,

$$\vec{T} = T\hat{x}_t = T\left(\cos\theta_t\,\hat{x}_r - \sin\theta_t\,\hat{z}_r\right), \tag{2.2.2}$$

with T the thrust magnitude (subsection 3.2); θ_t carries the whole misalignment.

Motion through the air produces an aerodynamic force and, about a reference point p (subsection 2.3), a pitching moment:

$$\vec{A}_f = -qS\left(C_x\hat{x}_r + C_z\hat{z}_r\right), \qquad \vec{A}_m = -qSc\,C_m\hat{y}_r, \qquad q = \tfrac{1}{2}\rho v_{rel}^2. \tag{2.2.3}$$

Here q is the dynamic pressure, S the reference area, c the reference length and C_x, C_z, C_m the coefficients, all supplied by subsection 3.4. The simulator's code calls C_m `C_m_code` internally, flagging the nose-down sign convention discussed in subsubsection 6.4.1.

2.3 Geometry

Each force acts at its own point on the rocket:

- A — nose tip, the origin of every length below;
- g — centre of gravity, where weight acts;
- t — thrust application point, where the line of thrust crosses the body;
- p — aerodynamic *moment reference* point, where $\vec{A}_f$ is taken to act and about which $\vec{A}_m$ is taken; the centre of pressure is the derived station $l_p - C_m\,c/C_z$ (subsection 6.3).

In $\mathcal{F}_r$, l is the axial distance of a point from A, positive towards the tail, and r its radial distance from the axis, along $-\hat{z}_r$:

$$\vec{Ag} = \begin{bmatrix} -l_g & 0 & -r_g \end{bmatrix}_r^\top, \qquad \vec{At} = \begin{bmatrix} -l_t & 0 & -r_t \end{bmatrix}_r^\top, \qquad \vec{Ap} = \begin{bmatrix} -l_p & 0 & 0 \end{bmatrix}_r^\top. \tag{2.3.1}$$

Moments are taken about g, so what they need are the arms from g to the other two points:

$$\vec{gt} = -\vec{Ag} + \vec{At} = \begin{bmatrix} l_g - l_t & 0 & r_g - r_t \end{bmatrix}_r^\top, \qquad \vec{gp} = -\vec{Ag} + \vec{Ap} = \begin{bmatrix} l_g - l_p & 0 & r_g \end{bmatrix}_r^\top. \tag{2.3.2}$$

The radial offsets r_g, r_t describe the axis misalignment of a real airframe. They are constant inputs, zero by default, like θ_t. The point p is a reference point, so $r_p \equiv 0$; any asymmetry of the external surfaces lives in the coefficients. The length l_t matters only for a canted thrust, through $\sin\theta_t$.

Figure (planned): Diagram (TikZ): the frames $\mathcal{F}_0$ and $\mathcal{F}_r$ with θ_r, the points A, g, p, t on the airframe, and the forces $\vec{W}$, $\vec{T}$, $\vec{A}_f$ with their arms.

2.4 Equations of motion

Translation follows from the sum of the three forces,

$$m\vec{a}_r = \vec{W} + \vec{T} + \vec{A}_f \tag{2.4.1}$$

$$= mg\hat{z}_g + T\left(\cos\theta_t\,\hat{x}_r - \sin\theta_t\,\hat{z}_r\right) - qS\left(C_x\hat{x}_r + C_z\hat{z}_r\right). \tag{2.4.2}$$

Rotation follows from the moments about g, along $\hat{y}_r$. Weight gives none:

$$I\varepsilon_r = \left[\vec{gt}\times\vec{T} + \vec{gp}\times\vec{A}_f + \vec{A}_m\right]\cdot\hat{y}_r \tag{2.4.3}$$

$$= T\left[(r_g - r_t)\cos\theta_t + (l_g - l_t)\sin\theta_t\right] + qS\left[-r_g C_x + (l_g - l_p)C_z - c\,C_m\right]. \tag{2.4.4}$$

Each term is a force component times its arm. For the aligned rocket, $r_g = r_t = \theta_t = 0$, thrust drops out and $I\varepsilon_r = qS\left[(l_g - l_p)C_z - c\,C_m\right]$.

The mass $m(t)$, inertia $I(t)$ and centre of gravity $l_g(t)$ change as propellant burns (subsection 3.3). Earth rotation adds one term to Equation 2.4.1 and none to Equation 2.4.3 (subsection 3.8). On the launch rail (subsection 4.1) and under a canopy (subsection 4.2) the attitude is frozen and Equation 2.4.1 is modified as described there. section 6 expands both equations by hand.

3 Models

Each subsection gives the interchangeable models behind one term of the equations of motion: the three forces (gravity, thrust with its mass, aerodynamics with the relative flow), the environment they depend on (atmosphere, wind), the random disturbances, the Earth-rotation correction, and the integration. The last subsection, sampled-data evaluation, changes when the models are evaluated and is off by default. The launch rail, recovery and air brake, which modify the equations, are in section 4.

3.1 Gravity

This subsection supplies the weight $\vec{W} = mg\hat{z}_g$: the magnitude g and the direction $\hat{z}_g$ (down, along the local vertical), chosen independently. Below, h is the altitude above the surface and R_e the Earth's mean radius; subsubsection 3.1.4 defines the altitudes.

3.1.1 Magnitude

- **Constant:** $g = g_0$.
- **Radial** (inverse-square, spherical non-rotating Earth).
- **Ellipsoidal** (WGS84, experimental): Somigliana surface gravity plus a second-order free-air correction, with ϕ the geodetic latitude of the launch site and constants from Appendix D.

$$g(r) = g_0 \left(\frac{R_e}{r}\right)^2 \qquad \text{(radial)} \tag{3.1.1}$$

$$\gamma_0(\phi) = \gamma_e \frac{1 + k\sin^2\phi}{\sqrt{1 - e^2\sin^2\phi}} \tag{3.1.2}$$

$$g(\phi, h) = \gamma_0 \left[1 - \frac{2}{a_e}\left(1 + f + m - 2f\sin^2\phi\right)h + \frac{3}{a_e^2}h^2\right]. \tag{3.1.3}$$

The ellipsoidal magnitude already contains the centrifugal part of the Earth's rotation.

Radius in the radial model. Here $r = |\vec{Og}|$ is the distance from the Earth's centre O to the rocket's centre of gravity g, and R_e is a constant. At launch

$$r(0) = R_0 = R_e + h_0 + h_{g,0}, \tag{3.1.4}$$

with h_0 the launch-site altitude above the mean Earth radius and $h_{g,0}$ the height of the initial centre of gravity above the ground (on the pad or rail), so that the initial position of g relative to O is consistent with the site. Later, $r = R_0 +$ (height gained), to first order in the curvature. The local radius of the real Earth is not constant: it varies with latitude, $R(\phi) \approx a_e(1 - f\sin^2\phi)$, by 0.3% between equator and pole, which is 0.7% in g; the ellipsoidal model above accounts for it. Ground-height models (terrain or geoid undulation) could replace h_0 and the flat ground by a position-dependent surface, as an option. (sim: uses a constant $R_e = 6371\,\text{km}$ and evaluates the radial g at $R_e + h_{AGL}$, with h_{AGL} measured from the launch position of the centre of gravity: neither h_0 nor $h_{g,0}$ enters (the ellipsoidal magnitude does include h_0, via the altitude above mean sea level))

3.1.2 Direction

- **Parallel** (default): $\hat{z}_g \equiv \hat{z}_0$, the local vertical at the launch point, for the whole flight.
- **Radial:** $\hat{z}_g$ points to the centre O of a sphere of radius R_e.
- **Ellipsoidal:** $\hat{z}_g$ is the inward normal of the WGS84 ellipsoid.

With $\vec{Og} = \vec{OO^*} + \vec{O^*g}$, $\vec{OO^*} = -R_e\hat{z}_0$ and $\vec{O^*g} = x\hat{x}_0 + z\hat{z}_0$ in the pitch plane,

$$\text{parallel:} \quad \hat{z}_g = \hat{z}_0, \tag{3.1.5}$$

$$\text{radial:} \quad \hat{z}_g = -\frac{\vec{Og}}{|\vec{Og}|} = -\sin\varphi\,\hat{x}_0 + \cos\varphi\,\hat{z}_0, \qquad \tan\varphi = \frac{x}{R_e - z}. \tag{3.1.6}$$

(sim: uses the arc angle $\varphi = x/R_e$ (Equation 3.1.9), which agrees with the exact angle at $z = 0$ and to first order elsewhere)

For the ellipsoid the normal does not pass through O: it makes the angle $\phi-\psi$ with $\vec{Og}$, where ψ is the geocentric latitude, $\tan\psi = (1-e^2)\tan\phi$ (at most 0.19°). The normal at the rocket is the normal at O^* turned by the change of geodetic latitude along the track, $\vartheta = \int_0^x \mathrm{d}x'/R_n(x')$, with R_n the local normal-curvature radius, so

$$\hat{z}_g = -\sin\vartheta\,\hat{x}_0 + \cos\vartheta\,\hat{z}_0. \tag{3.1.7}$$

Freezing R_n at its launch value gives the radius of the osculating sphere in the flight azimuth χ (Euler's formula, with M, N the meridional and prime-vertical radii of curvature),

$$R_\chi = \left(\frac{\cos^2\chi}{M(\phi_0)} + \frac{\sin^2\chi}{N(\phi_0)}\right)^{-1}, \quad M = \frac{a_e(1-e^2)}{(1-e^2\sin^2\phi)^{3/2}}, \quad N = \frac{a_e}{\sqrt{1-e^2\sin^2\phi}}, \tag{3.1.8}$$

so $\vartheta = x/R_\chi$, i.e. Equation 3.1.6 with $R_e \to R_\chi$. This differs from the exact normal at third order in ϑ, because R_n keeps changing with latitude along the track.

3.1.3 Vector field (magnitude + direction)

A single model may supply g and $\hat{z}_g$ together, as a vector field $\vec{g}(\vec{x}, t)$, so that $\vec{W} = m\vec{g}$. The radial model above is already the point-mass field; the refinements are

- **J2:** the second zonal harmonic (oblateness), $J_2 = 1.0826\times10^{-3}$. With the Earth's rotation it changes g by $\approx 0.53\%$ from equator to pole; the altitude effect is $\approx -0.3\%$ at 10 km (free-air gradient $-3.086\times10^{-6}\,\mathrm{s}^{-2}$).
- **Spherical-harmonic geopotential** (EGM2008 [20]), to a chosen degree and order, with local anomalies folded in: what a truncated global model misses.

The ellipsoidal magnitude above already contains the latitude variation; the remaining terms are negligible for a rocket flying below 10 km. (sim: neither exists; only the separate magnitude and direction models are implemented)

3.1.4 Position relative to O and true altitude

The non-parallel models need the altitude h_{true} over the curved ground, which the altitude-dependent models use in place of $-z$. The arc angle and the ground point under the rocket are

$$\varphi = \frac{x}{R_e}, \tag{3.1.9}$$

$$\vec{O^*P}(\varphi) = R_e\sin\varphi\,\hat{x}_0 + R_e(1-\cos\varphi)\,\hat{z}_0, \tag{3.1.10}$$

and the altitude is the offset of the rocket from that point along $-\hat{z}_g$,

$$h_{true}(x,z) = \left[x\hat{x}_0 + z\hat{z}_0 - \vec{O^*P}(\varphi)\right]\cdot\left(-\hat{z}_g\right) = x\sin\varphi - z\cos\varphi - R_e(1-\cos\varphi). \tag{3.1.11}$$

For small φ, $h_{true} \approx x^2/2R_e - z$. Impact is $h_{true} = 0$ instead of $z = 0$. For the ellipsoid, $R_e \to R_\chi$. Replacing R_e by R_0 would change the angles by a relative h_0/R_e, below 10^{-3} for a site under 6 km, so R_e is kept.

Figure (planned): Diagram (TikZ, exaggerated curvature): the Earth's centre O, launch site O^*, rocket position g, arc angle φ, the rotated $\hat{z}_g$ and the altitude h_{true} over the curved ground.

Which position matters depends on the question asked of the flight:

- **Relative to the launch point** (downrange x, height above the site): apogee, recovery and reporting.
- **Above mean sea level or the geoid:** atmosphere and regulations.
- **From the Earth's centre,** $r = R_e + h_0 + \ldots$**:** gravity.
- **Height above the local curved ground,** h_{true}**:** impact and terrain clearance.

The simulator integrates (x, z) relative to the launch point and reports the height above the ground h_{true} (equal to $-z$ for parallel gravity). The atmosphere uses $h_{true} + h_0$, the wind shear h_{true}.

Comparison (planned): constant, radial and ellipsoidal gravity (magnitude with the matching direction) on the demo rocket; the table gives apogee, downrange and flight time. Generated by the simulator.

3.2 Thrust

3.2.1 Magnitude

Thrust comes from the momentum of the ejected propellant and the pressure difference at the nozzle exit,

$$T = |\dot{m}_m|\, v_e + A_e(p_e - p_{atm}), \tag{3.2.1}$$

with $\dot{m}_m$ the motor mass flow, v_e the exhaust speed, p_e the exit pressure, A_e the exit area and p_{atm} the ambient pressure. In practice a static-test curve is used instead. The *reference* thrust $T_0(t)$ is given as points (t_i, T_i), $i = 1, \ldots, N$, from a motor file, interpolated linearly, held at T_1 for $t < t_1$ and zero for $t > t_N$; burnout is its falling zero-crossing:

$$T_0(t) = \mathrm{lerp}(t; \{t_i\}, \{T_i\})\,. \tag{3.2.2}$$

The test was run at a fixed ambient pressure p_0. While the motor is flowing ($T_0 > 0$), the thrust in flight is

$$T(t, h) = T_0(t) + A_e\,(p_0 - p_{atm}(h))\,, \tag{3.2.3}$$

which rises with altitude by at most $A_e p_0$ in vacuum; $A_e = 0$ recovers $T_0(t)$ exactly.

Comparison (planned): the bare thrust curve $T_0(t)$ against the pressure-corrected $T(t)$ ($A_e \neq 0$) for the demo motor on the demo rocket; the table gives apogee and burnout speed, the plot $T_0(t)$ and $T(t)$ along the flight, with burnout marked. Generated by the simulator.

3.2.2 Direction

The line of thrust may differ from the nose axis by an angle θ_t. Its unit vector $\hat{x}_t$ follows from $\hat{x}_r$ exactly as $\hat{x}_r$ does from $\hat{x}_0$:

$$\hat{x}_t = \cos\theta_t\,\hat{x}_r - \sin\theta_t\,\hat{z}_r, \qquad \hat{z}_t = \sin\theta_t\,\hat{x}_r + \cos\theta_t\,\hat{z}_r, \qquad \hat{y}_t \equiv \hat{y}_r, \tag{3.2.4}$$

so a positive θ_t tilts the thrust line the way a positive θ_r tilts the nose. Three options:

- **Ideal** (default): $\theta_t = 0$.
- **Misalignment**: θ_t is a constant input, a manufacturing-type error.
- **Controlled orientation** (gimbal): $\theta_t(t)$ commanded by a control law, the only pitch authority once off the rail. Future work.

The axial component is $T\cos\theta_t$ (4×10^{-5} lost at 0.5°), while $T\sin\theta_t$ acts at the arm $l_t - l_g$ and matters as a moment.

3.3 Mass, centre of gravity and moment of inertia

As propellant burns, mass leaves through the nozzle and both the centre of gravity and the moment of inertia change. The rocket is split into two bodies:

- the *dry rocket* (subscript r), everything that does not burn: constant mass m_r, centre of gravity $l_{g,r}$ from the nose tip A, inertia $I_{r,g}$ about its own centre of gravity;
- the *motor* (subscript m), casing and remaining propellant: mass $m_m(t)$, centre of gravity $l_{g,m}(t)$, inertia $I_{m,g}(t)$ about its own centre of gravity.

The subscript g in $I_{r,g}$, $I_{m,g}$ means "about that body's own centre of gravity"; the totals m, l_g, I carry no body subscript. t is time since ignition. The subscript p is not used for the propellant, as it names the aerodynamic reference point. g_0 is the standard gravity of the definition of specific impulse, unrelated to the local $g(h)$. (sim: one input g_0 serves for specific impulse, constant gravity and the radial reference; the values coincide ($9.80665\,\mathrm{m/s^2}$) unless a scenario overrides it)

3.3.1 Mass

The total mass is the dry rocket plus the motor,

$$m(t) = m_r + m_m(t). \tag{3.3.1}$$

Only m_m changes. It starts at m_0 and follows the mass flow until burnout, after which it is held at m_f (the casing and whatever else stays; $m_f = 0$ if m_m counts propellant only):

$$m_m(t) = m_0 + \int_0^t \dot{m}_m\,\mathrm{d}t', \qquad m_m \geq m_f. \tag{3.3.2}$$

The mass flow $\dot{m}_m < 0$ follows the thrust, as in the specific-impulse model below. (sim: the state m is integrated without a floor at m_f; only the propellant part is clamped at zero for the centre of gravity and inertia, so an ISP inconsistent with the thrust curve can burn below m_f)

Specific-impulse model. Inputs: the thrust curve and the specific impulse ISP (seconds). The propellant leaves at the rate that delivers exactly that thrust,

$$\dot{m}_m = -\frac{T_0(t)}{\mathrm{ISP}\; g_0}, \tag{3.3.3}$$

integrated until m_m reaches m_f. It uses the *reference* curve $T_0(t)$, not the pressure-corrected T, because ISP is defined from the stand measurement; the exit pressure term changes the force on the airframe, not the propellant consumption: the mass flow of a choked nozzle is set by the chamber conditions and the throat area, not by the ambient pressure [1]. If ISP is not given, the effective value $\mathrm{ISP} = J/(\Delta m\, g_0)$, with $J = \int T_0 \,\mathrm{d}t$ and $\Delta m = m_0 - m_f$, makes the integrated mass flow remove exactly Δm; the mass then decreases in proportion to the delivered impulse, which is the model used by OpenRocket and RocketPy [2, 3]. (sim: the mass flow is computed from the pressure-corrected T, not from the bare $T_0(t)$; the two differ only when $A_e \neq 0$ (default $A_e = 0$))

Comparison (planned): constant mass (launch values) against the specific-impulse mass model on the demo rocket; the table gives apogee, downrange and burnout speed, the plot total mass $m(t)$, centre of gravity $l_g(t)$ and moment of inertia $I(t)$ over the burn. Generated by the simulator.

3.3.2 Centre of gravity

The first moment of mass about the nose tip A is additive:

$$l_g(t)\, m(t) = l_{g,r}\, m_r + l_{g,m}(t)\, m_m(t) \quad \Longrightarrow \quad l_g(t) = \frac{l_{g,r}\, m_r + l_{g,m}(t)\, m_m(t)}{m(t)}. \tag{3.3.4}$$

The same holds for a non-zero radial offset,

$$r_g(t)\, m(t) = r_{g,r}\, m_r + r_{g,m}(t)\, m_m(t). \tag{3.3.5}$$

The simplest model of the motor's own centre of gravity is linear in the *burned* mass $m_b = m_0 - m_m$, with l_0, l_f its values at ignition and burnout and $\Delta m = m_0 - m_f$:

$$\frac{\partial l_{g,m}}{\partial m_b} = \frac{l_f - l_0}{\Delta m}, \qquad l_{g,m}(m_m) = l_0 + \frac{l_f - l_0}{\Delta m}\,(m_0 - m_m). \tag{3.3.6}$$

(With respect to m_m itself the sign of the derivative would flip.)

3.3.3 Moment of inertia

The inertias add once each body's own inertia is transferred to the total centre of gravity g by the parallel-axis theorem,

$$I(t) = I_r + I_m(t), \tag{3.3.7}$$

$$I_r = I_{r,g} + m_r\, d_r^2, \qquad I_m(t) = I_{m,g}(t) + m_m(t)\, d_m(t)^2, \tag{3.3.8}$$

with the transfer distances in the pitch plane

$$d_r^2 = \big(l_g(t) - l_{g,r}\big)^2 + \big(r_g(t) - r_{g,r}\big)^2, \qquad d_m^2 = \big(l_g(t) - l_{g,m}(t)\big)^2 + \big(r_g(t) - r_{g,m}(t)\big)^2. \tag{3.3.9}$$

On the axis ($r \equiv 0$) only the axial terms remain. I_r depends on time through $l_g(t)$ although $I_{r,g}$ and m_r do not.

The motor's own inertia is modelled like its centre of gravity, linear in burned mass, with I_0, I_f its values at ignition and burnout:

$$\frac{\partial I_{m,g}}{\partial m_b} = \frac{I_f - I_0}{\Delta m}, \qquad I_{m,g}(m_m) = I_0 + \frac{I_f - I_0}{\Delta m}\,(m_0 - m_m). \tag{3.3.10}$$

Radial offsets. The report treats r_g as a constant as-built input (zero by default). (sim: Equation 3.3.5 and Equation 3.3.9 make $r_g(t)$ time-dependent and add a radial term to the parallel-axis distance; the code keeps r_g (`u_B`) constant and omits the radial term. Exact for $r \equiv 0$, the default)

Variable-mass terms. The exhaust carries angular momentum out of the rocket, which acts as a damping (jet-damping) moment [22]. With $I\,\varepsilon_r$ (not $\mathrm{d}(I\omega_r)/\mathrm{d}t$ on the left, the $\omega_r\,\mathrm{d}I/\mathrm{d}t$ terms drop out and the planar equation is

$$I\,\varepsilon_r = \sum M - |\dot{m}_m|\,l_e^2\,\omega_r, \tag{3.3.11}$$

with $l_e \approx l_t - l_g$ the distance from the centre of gravity to the nozzle exit. The lever arm is disputed in the literature (exit, throat, mass-flow centre) [22]. Aerodynamic damping dominates except at low speed and high mass flow [6, 7]; RASAero and RockSim include jet damping. (sim: the jet-damping term of Equation 3.3.11 is not in the simulator; $I\,\varepsilon_r = \sum M$) (research: lever arm of the jet term, and the size of the term for the project's motors, from their $|\dot{m}_m|$, $l_t - l_g$ and $C_{m,q}$) (sim: the motor is a fixed-centroid propellant lump whose centre of gravity and inertia agree with the linear models (Equation 3.3.6, Equation 3.3.10) only at ignition and burnout) A mechanism (for instance a deployable surface) is a moving rigid sub-body: it enters Equation 3.3.4 and Equation 3.3.7 with its own time-dependent position, and makes $\mathrm{d}I/\mathrm{d}t$ non-zero. (research: no documented treatment of time-dependent mass properties from deployables found in OpenRocket, RockSim or RASAero)

Dry-rocket inputs from components. As in OpenRocket [4], the dry-rocket inputs can be built from the components i, each with mass m_i (measured, or density times volume), centre of gravity l_i and own inertia I_i:

$$m_r = \sum_i m_i, \qquad l_{g,r} = \frac{\sum_i m_i l_i}{m_r}, \qquad I_{r,g} = \sum_i \left(I_i + m_i(l_i - l_{g,r})^2\right). \tag{3.3.12}$$

3.3.4 Propellant grain geometry

A solid motor's $m_m(t)$, $l_{g,m}(t)$ and $I_{m,g}(t)$ follow from the receding grain. The propellant burns normal to its surface at the rate $r = a_b\,p_c^n$ ($n < 1$ for stable burning), and the burning area A_b over the throat area A_t is $K_n = A_b/A_t$. Equating the mass generated, $\rho_{pr} A_b\,a_b p_c^n$, to the mass discharged, $p_c A_t/c^*$, gives

$$p_c = \left(a_b\,\rho_{pr}\,c^* K_n\right)^{1/(1-n)}, \qquad \dot{m}_m = -\rho_{pr} A_b\,r, \tag{3.3.13}$$

with ρ_{pr} the propellant density and c^* the characteristic velocity [23]. The web regression $\mathrm{d}x/\mathrm{d}t = r$ is integrated until the web is consumed. For N BATES segments (outer surface inhibited, bore diameter $d = d_0 + 2x$, length $L = L_0 - 2x$, outer diameter D):

$$A_b(x) = N\left[\tfrac{\pi}{2}\left(D^2 - d^2\right) + \pi\,d\,L\right], \tag{3.3.14}$$

with the maximum at $x = (L_0 - 2d_0)/6$. A segment has mass $m = \rho_{pr}\tfrac{\pi}{4}(D^2 - d^2)L$ and inertias $I_{ax} = \tfrac{1}{8}m(D^2 + d^2)$, $I_{tr} = \tfrac{m}{12}\left[\tfrac{3}{4}(D^2 + d^2) + L^2\right]$ about its own centre. Both ends regress equally, so each segment's centre of gravity stays fixed; the stack follows from the segment masses and the parallel-axis theorem. The simulator would need a_b, n, ρ_{pr}, c^*, the segment geometry and the throat and exit areas [24]. (sim: the simulator has no grain model; $m_m(t)$ follows the thrust curve) (research: star and finocyl grains have no closed form; openMotor regresses them numerically by level-set / fast-marching [24])

3.4 Aerodynamics

3.4.1 Relative velocity and angle of attack

With $\vec{v}_w$ the wind velocity in $\mathcal{F}_0$ (subsection 3.6), the rocket's velocity relative to the air is

$$\vec{v}_{rel} = \vec{v}_r - \vec{v}_w = v_{rel,x}\hat{x}_r + v_{rel,z}\hat{z}_r, \tag{3.4.1}$$

with $v_{rel,x}$ the axial and $v_{rel,z}$ the normal component in $\mathcal{F}_r$. The airspeed and the angle of attack, taken in the body frame in the range $(-180°, 180°]$, are

$$v_{rel} = \sqrt{v_{rel,x}^2 + v_{rel,z}^2}, \qquad \alpha = \operatorname{atan2}(v_{rel,z}, v_{rel,x}). \tag{3.4.2}$$

A rocket flying nose-first has $\alpha \approx 0$, ascending or descending; a tail-first one has $\alpha \approx 180°$. They feed q and, through α, the coefficient lookup.

3.4.2 Force and moment

The force $\vec{A}_f$ and moment $\vec{A}_m$ of Equation 2.2.3 are *definitions*, not a model: they hold for every coefficient model, and the force is never supplied directly. S is the cross-section of the body tube and c its diameter. One pair (S, c) per component is shared by all of its coefficient models, since a coefficient means nothing except relative to the pair it was normalised by.

Multi-component aerodynamics. When the airframe is built from several components k (body, fins, nose cone, ...), each with its own reference point pp_k, force $\vec{A}_{f,k}$ and moment $\vec{A}_{m,k}$ about pp_k, the total force is the sum and the total moment about p adds the moment of each force about p:

$$\vec{A}_f = \sum_k \vec{A}_{f,k}, \qquad \vec{A}_m(p) = \sum_k \vec{A}_{m,k} + \sum_k p\vec{p}p_k \times \vec{A}_{f,k}, \tag{3.4.3}$$

where $p\vec{p}p_k = -\vec{A}p + A\vec{p}p_k$ is the position of pp_k relative to p, the arm of the force acting at pp_k. Each component is normalised by its own reference area and length, $\vec{A}_{f,k} = -qS_k(C_{x,k}\hat{x}_r + C_{z,k}\hat{z}_r)$ and $\vec{A}_{m,k} = -qS_k c_k\, C_{m,k}\hat{y}_r$, so S_k, c_k generally differ from the airframe's S, c. (sim: only a single aerodynamic component (one S, c and one coefficient set per airframe) is implemented; Equation 3.4.3 is not)

$l_p - l_g$ is the axial distance from g to p, positive with p aft, so that $\vec{gp} = -(l_p - l_g)\hat{x}_r$ and the caliber static margin is $\mathrm{SM} = (l_p - l_g)/c$.

Axial force and the flow direction. The axial force must oppose the axial component of the relative velocity, $v_{rel,x} = \hat{x}_r \cdot \vec{v}_{rel}$. In the body-axis form $-qSC_x\hat{x}_r$ with $C_x > 0$ it always points aft, so it would push a tail-first rocket faster. The coefficient is therefore multiplied by

$$f(\alpha) = \mathrm{clip}\Big(\frac{\cos\alpha}{\cos 85°},\, -1,\, 1\Big), \tag{3.4.4}$$

a smooth sign of the axial component: $\approx +1$ nose-first, ≈ -1 tail-first, and linear through zero between 85° and 95° so that the force does not jump. In Equation 2.2.3 C_x stands for $f\,C_x$. A table that itself covers $|\alpha| > 90°$ is used as it is. (sim: $f(\alpha)$ fixes only the sign of the axial force, with a linear ramp near 90°; the magnitude law of Equation 3.4.5 is not implemented)

Large angles of attack. CFD and wind-tunnel tables will not cover the full α range, and the drag *magnitude*, not only its sign, changes with α. Jorgensen's semi-empirical model for bodies of revolution [21] scales the axial coefficient and adds a viscous crossflow term to the normal force,

$$C_x(\alpha) = \begin{cases} C_x(0)\cos^2\alpha, & 0 \le \alpha \le 90°, \\ C_x(180°)\cos^2(180° - \alpha), & 90° < \alpha \le 180°, \end{cases} \tag{3.4.5}$$

$$C_z = \frac{S_{base}}{S}\sin 2\alpha\,\cos\frac{\alpha}{2} + \eta\, C_{d,n}\,\frac{S_p}{S}\sin^2\alpha, \tag{3.4.6}$$

with S_{base} the base area, S_p the planform area, η the finite-length factor (ratio of the drag of a finite cylinder to that of an infinite one) and $C_{d,n}$ the crossflow drag coefficient, evaluated at $M\sin\alpha$ and $Re\sin\alpha$ (1.2 at low crossflow Mach number). The first term of Equation 3.4.6 is slender-body potential flow, the second is the viscous crossflow. The method is verified mainly up to about 20°, and over the full range only at a few Mach numbers [21]. In practice the table is used at small α and a semi-empirical model beyond the table's range, as in OpenRocket and RASAero [5, 16]. (research: blending the table with the large-angle model at the table's edge is not standardised in the literature found; the sign convention of C_z for $\alpha > 90°$ is to be taken from [21])

3.4.3 Dynamic pressure

$$q = \tfrac{1}{2}\rho v_{rel}^2, \tag{3.4.7}$$

with ρ the local air density and v_{rel} the airspeed above.

3.4.4 Coefficients

The coefficients C_x, C_z, C_m and the centre of pressure l_p are not derived from the airframe's shape; they are supplied. They depend on the angle of attack α, the Mach number M and the Reynolds number Re (subsubsection 3.5.3). The pitch rate enters through the non-dimensional $\hat{\omega} = \omega_r c/(2v_{rel})$. The rate of change of the angle of attack is in practice lumped with it, since forced-oscillation tests and CFD give only the sum of the two derivatives [19]. Power-on or power-off (base drag) and surface roughness enter as discrete or parametric inputs, i.e. extra table axes or offsets [16]; accelerations are not used as inputs in standard practice [18, 16]. Three interchangeable models supply them, selected per run, in order of increasing fidelity; C stands for any of C_x, C_z, C_m.

- **Constant**: independent of the flight state, with l_p fixed. Useful for first-cut sizing and for isolating one effect in a sweep.

$$C = C_0 \tag{3.4.8}$$

- **Linearized**, about a reference condition in α, M, Re and the non-dimensional pitch rate, with l_p fixed; valid for small α. The derivatives with respect to $\hat{\omega}$ are $C_{m,q}$ and $C_{z,q}$.

$$C(\alpha, M, Re, \hat{\omega}) = C_0 + \alpha\frac{\partial C}{\partial \alpha} + M\frac{\partial C}{\partial M} + Re\frac{\partial C}{\partial Re} + \hat{\omega}\frac{\partial C}{\partial \hat{\omega}} \tag{3.4.9}$$

  If C_x must grow with α, the consistent body-axis option is a quadratic term, $C_x = C_{x,0} + C_{x,\alpha^2}\,\alpha^2$; the drag polar $C_D(C_L)$ is a wind-axis form for lifting bodies and would count again the drag already contained in $C_x(\alpha)$.

- **Lookup table**: tabulated over (α, M) from CFD or wind-tunnel data and interpolated at run time. The table gives the centre of pressure l_p from the nose, not the arm: CFD and wind-tunnel data know nothing of the centre of gravity, and the arm $l_p - l_g$ is formed from the live $l_g(t)$. This is the only model in which the static margin responds to the flight state.

$$\big(C_x,\, C_z,\, C_m,\, l_p\big) = \mathrm{interp}\big(\alpha, M;\ \mathrm{table}\big) \tag{3.4.10}$$

(sim: the linearized model has the α slope only (no M or Re slope), with $C_{z,0} = C_{m,0} = 0$, and uses $\sin\alpha$ in place of α for C_z, C_m; Re enters no model) (sim: the table is a two-variable (M, α) table that stores the arm $l_p - l_g$ (column `l1_m`) instead of l_p; the constant and linearized models also hold the arm fixed)

Figure (planned): Plot (from the example aerodynamic table): C_x, C_z, C_m against α for several Mach numbers, and C_x against M at $\alpha = 0$.

Comparison (planned): constant, linearized and table models on the demo rocket, with the coefficients of the linearized and constant models taken from the table; the table gives apogee, downrange, static margin at burnout and maximum α, the plot the three models overlaid on one coefficient. Generated by the simulator.

Pitch damping. Under all three models the pitch rate enters the moment through $C_{m,q}$,

$$\vec{A}_m = -qSc\,(C_m + C_{m,q}\,\hat{\omega})\,\hat{y}_r, \tag{3.4.11}$$

and $C_{m,q} > 0$ damps in this sign convention. A component build-up makes $C_{m,q}$ depend on Mach number, and in general on α and Mach number: Missile DATCOM tabulates $C_{m,q}(\alpha, M)$ [18].

The Table database has to cover α, M, Re and the geometry (fin planform and cant, body–fin interference, surface finish). A control surface (air brake, aileron, ...) has a defined point of entry: a control law perturbing C_x, C_z, C_m or l_p, or a scaled component of Equation 3.4.3. A deflection angle is then one more table axis, as in Missile DATCOM, which tabulates over α, Mach number, altitude, sideslip and fin deflections [18]. (research: project definitions, not literature: the CFD/data-extraction pipeline that populates the database, the control surface's geometry and its deflection law)

3.5 Atmosphere

Provides the air density ρ, speed of sound a and dynamic viscosity μ at altitude h (and horizontal position), which form the Reynolds and Mach numbers. Turbulence and gusts are in subsection 3.7.

3.5.1 Model tiers

Three tiers, one interface (ρ, T, p, a, μ):

1. **ISA**: standard layered atmosphere (Table 2).
2. **Sounding**: one measured (T, p) profile, interpolated in altitude.
3. **Field**: several stations, interpolated in altitude and horizontal position. (sim: interpolation in time between soundings is not implemented)

Comparison (planned): ISA, one sounding (a demo station from the data folder) and the multi-station field on the demo rocket; the table gives apogee, downrange and maximum dynamic pressure, the plot T, p, ρ against altitude for ISA and the sounding, the difference shaded. Generated by the simulator.

ISA. Geopotential altitude from the geometric altitude above mean sea level h_{msl}:

$$H = \frac{R_{isa} h_{msl}}{R_{isa} + h_{msl}}, \qquad R_{isa} = 6356766\,\mathrm{m}. \tag{3.5.1}$$

Within the layer of Table 2 containing H, temperature is linear in geopotential altitude:

$$T = T_b + L(H - H_b). \tag{3.5.2}$$

For a non-zero lapse rate the pressure follows the polytropic relation,

$$p = p_b (T/T_b)^{-g_{isa}/(L R_d)}, \tag{3.5.3}$$

and for an isothermal layer ($L = 0$) the exponential form,

$$p = p_b \exp\left(\frac{-g_{isa}(H - H_b)}{R_d T_b}\right). \tag{3.5.4}$$

Density, speed of sound and viscosity follow from the ideal gas law and Sutherland's law:

$$\rho = \frac{p}{R_d T}, \qquad a = \sqrt{\gamma R_d T}, \qquad \mu = \mu_0 \left(\frac{T}{T_0}\right)^{3/2} \frac{T_0 + S_\mu}{T + S_\mu}, \tag{3.5.5}$$

with $R_d = 287.0528\,\mathrm{J/(kg\,K)}$, $\gamma = 1.4$, $g_{isa} = 9.80665\,\mathrm{m/s^2}$ (the ISA's own fixed value), $\mu_0 = 1.716 \times 10^{-5}\,\mathrm{Pa\,s}$, $T_0 = 273.15\,\mathrm{K}$, $S_\mu = 110.4\,\mathrm{K}$.

Sounding. One station's measured (h_{msl}, p, T, u, v) profile, with temperature and wind interpolated linearly and pressure log-linearly:

$$T(h) = \mathrm{lerp}(h; \{h_i\}, \{T_i\}), \quad p(h) = \exp \mathrm{lerp}(h; \{h_i\}, \{\ln p_i\}), \quad u(h), v(h) = \mathrm{lerp}(h; \{h_i\}, \{u_i, v_i\}). \tag{3.5.6}$$

ρ, a, μ follow from the interpolated T, p as for ISA. Below the lowest level the profile continues with the sounding's own lapse rate; above the highest, an ISA-shaped profile re-anchored at the top level keeps T, p continuous; wind is held at the edge value.

Field. N stations are projected onto a tangent plane centred at the launch site (ϕ_0, λ_0), together with the rocket's ground position $(x \sin\chi, x\cos\chi)$:

$$E = R_e \cos\phi_0\,(\lambda - \lambda_0), \qquad N = R_e\,(\phi - \phi_0). \tag{3.5.7}$$

The N profiles are combined, weighted by inverse squared distance d_i, into one profile:

$$w_i = \frac{1}{d_i^2 + \varepsilon^2}, \qquad \hat{w}_i = \frac{w_i}{\sum_j w_j}, \qquad (T, \ln p, u, v) = \sum_i \hat{w}_i\,(T_i, \ln p_i, u_i, v_i), \tag{3.5.8}$$

then interpolated in altitude as in the sounding tier. A single station reduces every $\hat{w}_i$ to 1 and reproduces the sounding tier. (sim: each station is first resampled onto a common altitude grid (0 to 30 km in 100 m steps) before weighting, so a single station reproduces the sounding tier only to the error of that grid interpolation)

Inverse-distance weighting is defined everywhere, also outside the polygon the stations span, and never oscillates; ε (a few hundred metres) removes the singularity at zero distance. A thin-plate-spline radial basis function is available as an alternative.

3.5.2 Density

Water vapour is lighter than dry air ($M_v = 18.02$ against $M_d = 28.96\,\mathrm{g/mol}$), so at a given pressure and temperature damp air is *less* dense. The effect is at the percent level, and radiosondes report the dewpoint needed to correct for it.

Partial pressures. Treat the mixture as two ideal gases sharing a volume, dry air at partial pressure $p - e$ and vapour at e:

$$\rho = \frac{p - e}{R_d T} + \frac{e}{R_v T} = \frac{p}{R_d T}\left[1 - \frac{e}{p}(1 - \epsilon)\right], \qquad \epsilon = \frac{R_d}{R_v} = \frac{287.0528}{461.5} = 0.6220. \tag{3.5.9}$$

The bracket is always below one. Folding it into an effective *virtual temperature*, the temperature dry air would need to have the moist density at the same pressure, keeps the dry-air gas constant R_d in every formula:

$$T_v = \frac{T}{1 - \frac{e}{p}(1 - \epsilon)}, \qquad \rho = \frac{p}{R_d T_v}, \tag{3.5.10}$$

with $T_v \geq T$, equality for dry air.

Vapour pressure from dewpoint. The dewpoint T_d is the temperature to which the parcel must be cooled, at constant pressure and mixing ratio, to reach saturation, so the vapour pressure is the saturation pressure at the dewpoint, $e = e_s(T_d)$. For e_s the Magnus form is used, with the Alduchov–Eskridge coefficients, T_d in °C and e_s in hPa:

$$e_s(T_d) = 6.1094 \exp\left(\frac{17.625\,T_d}{T_d + 243.04}\right). \tag{3.5.11}$$

It is accurate to better than 0.4% over −40 to +50 °C. The factor of 100 converting hPa to Pa before forming e/p is the easy mistake.

The correction is an opt-in switch, off by default (dry air, $\rho = p/(R_d T)$), and applies to the sounding and field tiers. At a warm, humid surface ($T = 30$ °C, $T_d = 26$ °C) it changes the density by −1.25%, at a temperate one ($T = 14.2$ °C, $T_d = 10$ °C) by −0.48%, near the tropopause by less than 10^{-5}. The speed of sound and viscosity are not corrected ($\sim 0.2\%$).

3.5.3 Reynolds and Mach numbers

$$Re = \frac{\rho v_{rel} l_{Re}}{\mu}, \qquad M = \frac{v_{rel}}{a}, \tag{3.5.12}$$

with v_{rel} the airspeed, l_{Re} the Reynolds reference length, a separate input rather than the body diameter c, μ the dynamic viscosity and a the local speed of sound. No coefficient model uses Re yet.

3.6 Wind

Goal: given time t and position (altitude h, and horizontal position for the interpolated tier), provide the wind velocity $\vec{v}_w$ in $\mathcal{F}_0$. The aerodynamic force and moment see the air only through $\vec{v}_{rel} = \vec{v}_r - \vec{v}_w$. The tiers mirror the atmosphere:

1. **Simple analytic model**: constant wind plus an altitude-dependent shear profile, with an optional turbulence term (subsection 3.7).
2. **Real data** (one sounding): a measured or forecast profile $\vec{v}_w(h)$ at one location and time, or an average for that location.
3. **Real data across multiple stations**: profiles from several locations, interpolated in altitude and horizontal position by inverse-distance weighting, as for the atmosphere (`apps/sim/engine/atmosphere.py`). (sim: interpolation in time between soundings is not implemented)

Flight-plane projection. The launch heading χ fixes the 2D flight plane. A horizontal wind given as ENU components (u, v), or as meteorological speed and direction (W, ψ) with $u = -W\sin\psi$, $v = -W\cos\psi$ (ψ the direction the wind blows *from*), projects onto it as

$$w_x = u\sin\chi + v\cos\chi = -W\cos(\psi - \chi), \qquad w_y = u\cos\chi - v\sin\chi = -W\sin(\psi - \chi), \tag{3.6.1}$$

with w_x the downrange component feeding $\vec{v}_w$ and $w_z = 0$. The cross-plane component w_y is computed and discarded: crosswind weathercocking needs a full 6DOF model.

Analytic. Power-law shear with constant direction:

$$W(h) = W_{ref}\left(\max(h, h_{ref})/h_{ref}\right)^n, \qquad \psi = \psi_0. \tag{3.6.2}$$

h is the altitude above the local ground and $n \approx 1/7$ the shear exponent (open terrain; lower for smoother, up to ~ 0.4 for rougher terrain). It is valid within the surface layer only — the lowest few hundred metres to 1–2 km — above which the sounding tiers apply. This is the *mean* profile; the fluctuation about it is in subsection 3.7.

Figure (planned): Plot (simulator output): the power-law mean wind $W(h)$ for two shear exponents, and a sampled Dryden turbulence realisation $u(t)$ about it.

Comparison (planned): no wind, the analytic power-law mean profile and a sounding profile on the demo rocket (demo station from the data folder); the table gives apogee, downrange, landing point and maximum angle of attack. Generated by the simulator.

Sounding / field. The same measured (u, v) profiles and interpolation as the atmosphere, projected through the formula above; never interpolated as (W, ψ), since direction wraps at 360°.

3.6.1 Vertical wind

All tiers return horizontal wind only ($w_z = 0$). The structure admits a vertical component: $\vec{v}_w = w_x\hat{x}_0 + w_z\hat{z}_0$, so an updraft of speed w_{up} is $w_z = -w_{up}$ and changes $\vec{v}_{rel}$, hence v_{rel} and α, directly. It matters most where v_{rel} is small (near the pad, around apogee, under canopy), since it rotates the relative wind by about w_{up}/v_{rel}. Three models could supply it:

- **Orographic lift**, $w \approx \vec{v}_h \cdot \nabla h_{terrain}$. (research: needs a terrain elevation model (DEM) of the site)
- **Thermal updraft** parameterisation from the soaring literature, e.g. the convective velocity scale w_* from the surface heat flux and boundary-layer depth. (research: needs heat-flux or insolation data)
- **Measured profile**: a sounding balloon carrying a 3-axis pitot or anemometer gives $w_z(h)$ directly. (research: feasibility: the balloon's own vertical motion must be subtracted, and sensor accuracy at small w)

(sim: $w_z = 0$ in all tiers; no vertical wind is implemented)

3.7 Noise and disturbances

Two random effects perturb the flight about the deterministic models: the motor's thrust and the air's motion. Both are optional and off by default.

3.7.1 Thrust noise

Even a perfectly aligned motor does not push exactly along its axis. The flow through the nozzle produces a small, random side force, from jet and throat asymmetry, nozzle flow separation, combustion instability and exhaust swirl, usually a small fraction of T. It is modelled along the thrust-frame normal $\hat{z}_t$ (Equation 3.2.4),

$$\vec{T}_s(t) = \kappa\, T(t)\, \xi(t)\, \hat{z}_t, \tag{3.7.1}$$

with κ the intensity as a fraction of thrust and $\xi(t)$ a zero-mean unit-variance random process with a correlation time τ_c. It enters the equations of motion as part of the thrust, applied at t, so it also adds a moment about g. The same flow produces a motor roll torque about $\hat{x}_r$ (internal vortex flow, nozzle ablation), which has no effect on the pitch plane but is a term of the roll moment L_x of subsection 4.4 whenever that freedom is enabled. It is larger for star or finned grains, appears while the slotted part burns, and has high peaks but small angular impulse; peaks up to 163 N m were measured on SCOUT stages, and no prediction method is known [9]. (research: roll-torque magnitude and model for small motors: no $\kappa T\xi$-type model in the literature) For large solid motors the thrust-vector misalignment is quoted as a fraction of a degree (0.1° to 0.5°, i.e. $\kappa \approx \tan\delta \approx 2\times10^{-3}$ to 9×10^{-3}), and its time structure is a random, nearly constant bias plus short transients at ignition and tail-off rather than a stationary process [8]. (research: not implemented; τ_c and the roll-torque level are not in the literature found, and the values above are for large upper-stage motors, not small motors: need test-stand data)

3.7.2 Atmospheric disturbances

The mean wind of subsection 3.6 is deterministic; turbulence is a stationary random fluctuation about it, known only through its statistics (intensity σ, length scale L). It perturbs the air's velocity, so it belongs to the wind. Two variants, selected independently of the mean profile: **none** (default) and **Dryden**, the MIL-F-8785C [10] continuous-turbulence spectrum, on the analytic tier only. (sim: a discrete-gust model, as a deterministic wind increment $\Delta\vec{v}_w(t)$, is not implemented)

The spectrum has three components: longitudinal u (along the mean wind), lateral v (horizontal, across it) and vertical w. In the flight plane w always acts and u acts through its projection; v is out of plane when the mean wind lies in the plane, and is carried for the 3D model. (sim: only the horizontal u and v are implemented; the vertical component w, which is in-plane, is missing. Low altitude, MIL-F-8785C: $L_w = h$, $\sigma_w = 0.1W_{20}$, Φ_w of the form of Φ_v with (L_w, σ_w)) MIL-HDBK-1797 uses instead $L_w = h/2$, $L_v = L_u/2$ with a matching PSD form; the two conventions must not be mixed, and this report keeps MIL-F-8785C.

The scales and intensities are the low-altitude formulas, in feet ($h_{ft} = h/0.3048$, h the altitude above ground, clamped at zero), with W_{20} the mean profile Equation 3.6.2 at 20 ft:

$$L_u = L_v = \frac{h_{ft}}{(0.177 + 0.000823\,h_{ft})^{1.2}}, \quad L_w = h_{ft}, \qquad \sigma_w = 0.1\,W_{20}, \quad \sigma_u = \sigma_v = \frac{\sigma_w}{(0.177 + 0.000823\,h_{ft})^{0.4}}. \tag{3.7.2}$$

Three bands, the middle one an interpolation across the gap the spec leaves open:

Band	Condition	L [ft]	σ
low	$h_{ft} \le 1000$	Equation 3.7.2	Equation 3.7.2
transition	$1000 < h_{ft} < 2000$	linear to 1750	linear to σ_{high}
isotropic	$h_{ft} \ge 2000$	1750, all components	σ_{high} (an input)

The one-sided temporal power spectral densities, normalised to $\int_0^\infty \Phi\,\mathrm{d}\omega = \sigma^2$, are

$$\Phi_u(\omega) = \sigma_u^2 \frac{2L_u}{\pi v_{ref}} \frac{1}{1 + (L_u\omega/v_{ref})^2} \tag{3.7.3}$$

$$\Phi_v(\omega) = \sigma_v^2 \frac{L_v}{\pi v_{ref}} \frac{1 + 3\,(L_v\omega/v_{ref})^2}{\left[1 + (L_v\omega/v_{ref})^2\right]^2} \tag{3.7.4}$$

with v_{ref} a fixed reference airspeed. A realisation is a frozen random-phase sum over a log-spaced grid $\{\omega_i\}$ (periods 600 s to 0.2 s), with the amplitudes renormalised so that $\mathrm{Var}[u] = \sigma_u^2$ exactly:

$$k_i = \Phi(\omega_i)\Delta\omega_i, \qquad w_i = \sqrt{\frac{2k_i}{\sum_j k_j}}, \qquad u(t) = \sigma_u \sum_i w_i \sin(\omega_i t + \varphi_i), \tag{3.7.5}$$

and likewise for $v(t)$ with independent phases; the phases $\{\varphi_i\}$ are drawn once per seed.

Along the mean wind u and across it v compose with the flight-plane projection rather than adding to W as scalars. With $\hat{e}_u$ the unit vector along the mean flow, $\hat{e}_v$ it rotated by $+90°$ in the horizontal plane, and $\hat{y}_0$ the cross-plane axis,

$$\hat{e}_u = -\cos(\psi - \chi)\,\hat{x}_0 - \sin(\psi - \chi)\,\hat{y}_0, \qquad \hat{e}_v = \sin(\psi - \chi)\,\hat{x}_0 - \cos(\psi - \chi)\,\hat{y}_0, \tag{3.7.6}$$

$$(w_x, w_y) = (W + u)\,\hat{e}_u + v\,\hat{e}_v, \tag{3.7.7}$$

which reduces to the mean-only projection when $u = v = 0$ and is not clamped, so $W + u < 0$ is a gust reversing the local flow. The 2D closure discards w_y, so v has no effect on the trajectory. The perturbation is zero at $h = 0$, where $L = 0$.

Limitation: v_{ref} is a fixed input, not the rocket's v_{rel}, since a spectrum driven by the instantaneous airspeed would make the right-hand side depend on more than $(t, \vec{y})$ and break the adaptive solver. It sets only where the spectrum's knee sits in frequency, not the intensity.

Comparison (planned): Dryden turbulence off and on over an ensemble of seeds, on the demo rocket in the power-law mean wind; the table gives mean and spread of apogee and downrange, the plot the scatter of the landing point. Generated by the simulator.

3.8 Non-inertial frame corrections (Earth's rotation)

The WGS84 gravity magnitude already contains the centrifugal effect of the Earth's rotation at the launch site. Reinterpreting $\mathcal{F}_0$ as Earth-fixed adds two fictitious accelerations to the translational equation:

$$\vec{a}_{Coriolis} = -2\vec{\Omega} \times \vec{v}_r, \qquad \vec{a}_{centrifugal} = -\vec{\Omega} \times (\vec{\Omega} \times \vec{r}), \tag{3.8.1}$$

with $\vec{\Omega}$ the Earth's angular velocity ($|\vec{\Omega}| = 7.2921 \times 10^{-5}$ rad/s) and $\vec{r}, \vec{v}_r$ the rocket's position and velocity in the Earth-fixed frame. The Coriolis term depends on the flight's own velocity and heading, and is the one implemented.

Ground-frame basis. Work in the local East–North–Up triad $(\hat{E}, \hat{N}, \hat{U})$ at the launch site (ϕ_0, λ_0), with ϕ_0 the geodetic latitude (written ϕ in the gravity model). $\mathcal{F}_0$ is fixed by the launch heading χ (degrees clockwise from North, the direction of $+\hat{x}_0$) and $\hat{z}_0$ pointing *down*:

$$\hat{x}_0 = \sin\chi\,\hat{E} + \cos\chi\,\hat{N}, \qquad \hat{y}_0 = \cos\chi\,\hat{E} - \sin\chi\,\hat{N}, \qquad \hat{z}_0 = -\hat{U}. \tag{3.8.2}$$

The triad is right-handed, $\hat{x}_0 \times \hat{y}_0 = \hat{z}_0$, and $\hat{y}_0$ points to the *right* of the downrange direction. The decomposition of $\vec{\Omega}$ below is evaluated once, at the launch site, in this fixed frame; it does *not* follow a rotating $\hat{z}_g$.

Components of $\vec{\Omega}$. The rotation axis points to the celestial north pole, which at latitude ϕ_0 lies in the local meridian plane, tilted from the vertical by the co-latitude:

$$\vec{\Omega} = \Omega \left(\cos\phi_0 \, \hat{N} + \sin\phi_0 \, \hat{U} \right), \qquad \Omega = 7.2921 \times 10^{-5} \, \text{rad/s}. \tag{3.8.3}$$

Projecting onto the basis vectors,

$$\vec{\Omega} = \Omega \left(\cos\phi_0 \cos\chi \, \hat{x}_0 - \cos\phi_0 \sin\chi \, \hat{y}_0 - \sin\phi_0 \, \hat{z}_0 \right). \tag{3.8.4}$$

Which component matters in 2D. With $\vec{v}_r = v_x \hat{x}_0 + v_z \hat{z}_0$,

$$\vec{\Omega} \times \vec{v}_r = \Omega_y v_z \, \hat{x}_0 + \left(\Omega_z v_x - \Omega_x v_z \right) \hat{y}_0 - \Omega_y v_x \, \hat{z}_0, \tag{3.8.5}$$

so

$$\vec{a}_{Coriolis} = -2\vec{\Omega} \times \vec{v}_r = \underbrace{-2\Omega_y \left(v_z \hat{x}_0 - v_x \hat{z}_0 \right)}_{\text{in-plane}} \underbrace{- 2 \left(\Omega_z v_x - \Omega_x v_z \right) \hat{y}_0}_{\text{out-of-plane}}. \tag{3.8.6}$$

The in-plane acceleration comes *entirely* from Ω_y, the component normal to the flight plane; Ω_x and Ω_z, lying inside the plane, give a purely out-of-plane acceleration. Projecting $\vec{\Omega}$ onto the plane would keep exactly the wrong half.

Simplification (2D closure). Only the in-plane bracket is retained. This is not a smallness argument: the discarded lateral acceleration is of the same order, and for some geometries larger. It is dropped because the model has no lateral degree of freedom, the same closure as for the crosswind w_y, lifted by the same future 6DOF work. Consequently $\Omega_y = -\Omega \cos\phi_0 \sin\chi$ vanishes for a launch due North or South and at the poles, where the 2D model predicts *zero* Coriolis effect although the true effect is maximal and lateral. Substituting Ω_y,

$$\vec{a}^{\,2D}_{Coriolis} = 2\Omega \cos\phi_0 \sin\chi \left(v_z \, \hat{x}_0 - v_x \, \hat{z}_0 \right), \tag{3.8.7}$$

that is, with $v_z < 0$ for an ascending rocket,

$$a_{Coriolis,x} = +2\Omega \cos\phi_0 \sin\chi \; v_z, \qquad a_{Coriolis,z} = -2\Omega \cos\phi_0 \sin\chi \; v_x. \tag{3.8.8}$$

At the equator with $\hat{x}_0$ East this gives the westward drift of a rising rocket and the Eötvös lift of an eastward one.

Residual centrifugal term. Write the geocentric position as $\vec{r} = \vec{R}_0 + \vec{s}$, with $\vec{R}_0$ from the Earth's centre to the launch site and $\vec{s} = x\hat{x}_0 + z\hat{z}_0$ the simulation's position. Since $-\vec{\Omega} \times (\vec{\Omega} \times \vec{a}) = \Omega^2 \vec{a} - (\vec{\Omega} \cdot \vec{a})\vec{\Omega}$ is linear in $\vec{a}$,

$$\vec{a}_{centrifugal} = \underbrace{\Omega^2 \vec{R}_0 - (\vec{\Omega} \cdot \vec{R}_0)\vec{\Omega}}_{\text{constant, already in } \gamma_0(\phi)} + \underbrace{\Omega^2 \vec{s} - (\vec{\Omega} \cdot \vec{s})\vec{\Omega}}_{\text{residual}}. \tag{3.8.9}$$

The residual is bounded by $\Omega^2 |\vec{s}| = 5.3 \times 10^{-9} \, |\vec{s}|$, i.e. $5 \times 10^{-5} \, \text{m/s}^2$ at $|\vec{s}| = 10 \, \text{km}$, three orders below the Coriolis term, and is left unimplemented.

Moment. Both accelerations are body forces proportional to mass, so their resultant acts at g and gives *zero* moment about it. The only coupling is a gyroscopic torque from the body's pitch rate, of order $2I\Omega\omega_r$, i.e. about $1.5 \times 10^{-4} \, \text{rad/s}^2$ at $\omega_r = 1 \, \text{rad/s}$; it is dropped.

Result. The translational equation gains one term, evaluated directly in $\mathcal{F}_0$ like $\vec{W}$:

$$m\vec{a}_r = \vec{W} + \vec{T} + \vec{A}_f + m \, \vec{a}^{\,2D}_{Coriolis}. \tag{3.8.10}$$

It adds no parameter beyond ϕ_0 and χ. At $|\vec{v}_r| = 300 \, \text{m/s}$ it is at most $4.4 \times 10^{-2} \, \text{m/s}^2$ (equatorial East launch), about $2.8 \times 10^{-2} \, \text{m/s}^2$ at $\phi_0 = 50°$: 0.3% of g, or order 10 m of displacement over a 30 s powered ascent. This is the same order as the $g\,x/R_e \approx 1.5 \times 10^{-2} \, \text{m/s}^2$ (at $x = 10 \, \text{km}$) that gravity acquires along the rotated vertical; the two are *independent* and must not be multiplied together.

Comparison (planned): Earth-rotation correction off and on, at the demo launch site for several headings χ and for latitudes from the equator to 60°, on the demo rocket; the table gives the change in apogee, downrange and landing point. Generated by the simulator.

(sim: $\vec{\Omega}$ is resolved once at the launch site in $\mathcal{F}_0$ while gravity may follow the rotated $\hat{z}_g$; resolve it in the local frame along the trajectory, natural in a 3D/ECEF formulation (effect about 3×10^{-6} of the Coriolis term at 10 km))

3.9 States, initial conditions and integration

The rocket has two groups of kinematic states, resolved in the fixed frame $\mathcal{F}_0$:

- *translation*: position $\vec{x}_r = x\,\hat{x}_0 + z\,\hat{z}_0$, velocity $\vec{v}_r = v_x\,\hat{x}_0 + v_z\,\hat{z}_0$ and acceleration $\vec{a}_r$;
- *orientation*: pitch angle θ_r, pitch rate ω_r and angular acceleration ε_r.

Each level is the time derivative of the one below it, and the mass follows the motor mass flow:

$$\frac{\mathrm{d}\vec{x}_r}{\mathrm{d}t} = \vec{v}_r, \quad \frac{\mathrm{d}\vec{v}_r}{\mathrm{d}t} = \vec{a}_r, \qquad \frac{\mathrm{d}\theta_r}{\mathrm{d}t} = \omega_r, \quad \frac{\mathrm{d}\omega_r}{\mathrm{d}t} = \varepsilon_r, \qquad \dot{m} = \dot{m}_m = -\frac{T_0(t)}{\mathrm{ISP}\,g_0}. \tag{3.9.1}$$

The accelerations $\vec{a}_r$, ε_r are outputs of Equation 2.4.1 and Equation 2.4.3, evaluated at every call of the right-hand side, not states. The integrated state vector has dimension seven,

$$\vec{y} = (x,\ z,\ v_x,\ v_z,\ \theta_r,\ \omega_r,\ m)\,, \tag{3.9.2}$$

with z and v_z positive *down*. m is read only when the variable-mass model is on; otherwise m and I are fixed and $\dot{m} = 0$. The right-hand side has three modes, selected in this order: on the rail, under canopy, otherwise free flight. The mass flow is that of the specific-impulse model (subsubsection 3.3.1).

Initial conditions. The default start is at rest on the pad, at ignition, with the nose along the rail:

$$\vec{x}_r(0) = \vec{0}, \quad \vec{v}_r(0) = \vec{0}, \quad \theta_r(0) = \theta_{rail}, \quad \omega_r(0) = 0, \quad m(0) = m_{launch}, \tag{3.9.3}$$

with θ_{rail} the rail elevation (default 90°) and $m_{launch} = m_r + m_0$ the liftoff mass. Time starts at $t = 0$, the ignition instant of the thrust curve. The launch site supplies

- latitude ϕ_0 and heading χ — Earth-rotation correction and ellipsoidal gravity;
- elevation h_0 — atmosphere and gravity only; it never enters z;
- rail elevation and length — the launch rail.

A scenario may override the whole state; nothing checks that $\theta_r(0) = \theta_{rail}$. The models carry no hidden dynamical state, so they are pure functions of $(t, \vec{y})$.

Integrators. Two integrators are available behind one interface, chosen by a setting:

- **Adaptive Runge–Kutta 4(5)** (default): the SCIPY `RK45` scheme with relative tolerance 10^{-8} and absolute tolerance 10^{-10} on every component of $\vec{y}$, dense output on, and a time limit of 300 s. Events are located to solver tolerance by root finding on the dense output.
- **Fixed-step classical Runge–Kutta 4**, hand-written, with step h (default 1 ms):

$$\vec{y}_{k+1} = \vec{y}_k + \frac{h}{6}\left(\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4\right), \tag{3.9.4}$$

  with $\vec{k}_1 = \vec{f}(t_k, \vec{y}_k)$, $\vec{k}_{2,3} = \vec{f}\left(t_k + \frac{h}{2}, \vec{y}_k + \frac{h}{2}\vec{k}_{1,2}\right)$, $\vec{k}_4 = \vec{f}(t_k + h, \vec{y}_k + h\,\vec{k}_3)$ and $\vec{f}$ the right-hand side. It has no step control; events are detected afterwards from sign changes between steps and placed by linear interpolation, so they are accurate to about h. It is the cross-check of the adaptive path and the only one that supports the sampled-data mode.

Both integrate the same right-hand side and differ only in numerical error. (research: document the cost and accuracy of the two integrators with a convergence study (error against h and against tolerance))

Events. Five events are watched; only the first ends the run. Each is a sign change of a scalar function of $(t, \vec{y})$:

- **ground impact** (terminal): the ground depth crosses zero going down, z for flat ground, the curved-ground depth otherwise. Ignored for the first 0.1 s, since $z = 0$ at the start;
- **apogee**: v_z crosses from negative to positive in the launch frame, also ignored for the first 0.1 s;
- **burnout**: the thrust falls through a small threshold (10^{-9} N) from above, so a motor that never produces thrust never triggers it;
- **rail exit**: s_{rail} crosses L_{rail} upwards; never active when $L_{rail} = 0$, and also ignored for the first 0.1 s;

- **parachute deployment**: the deployment predicate turns true; never active without a recovery model.

Figure (planned): Plot (simulator output, a named demo scenario): altitude against downrange with the events marked — rail exit, burnout, apogee, parachute deployment, impact.

Apart from ground impact the events only report times. The rail, canopy and air-brake modes are state predicates evaluated inside the right-hand side, which cannot read an event not yet finalised; hence the same condition appears twice, as a predicate that acts and an event that reports. The apogee event is the launch-frame v_z crossing, not the maximum of the true altitude above a curved ground. (research: decision: keep the launch-frame v_z crossing, or redefine apogee as the maximum of the true altitude over curved ground)

Comparison (planned): adaptive RK45 against fixed-step RK4 (several h) on the demo rocket; the table gives apogee, burnout and impact times, downrange and run time, with the difference to the finest RK4 run. Generated by the simulator.

3.10 Sampled-data evaluation (zero-order hold)

This is an interchangeable *mode of evaluating* the models, changing *when* a quantity is computed, not *what* it is. By default every model is a pure function of $(t, \vec{y})$ (Equation 3.9.2), re-evaluated at every right-hand-side call. A flight computer's measure–compute–hold cycle, an integral term or a filter with internal state cannot be expressed that way; this mode makes it expressible.

With a sample rate $f_s > 0$ and period $\tau_s = 1/f_s$, the ticks are $t_k = k\tau_s$. At each tick every registered *channel* — a named sampled quantity — is evaluated once, at that tick's state, and held:

$$c(t) = c_k \equiv c(t_k, \vec{y}(t_k);\, c_{k-1})\,, \qquad t \in [t_k, t_{k+1}). \tag{3.10.1}$$

A channel receives *its own* previous held value c_{k-1} (undefined at the first tick). That makes it recursive, and so an integral accumulator, a rate limiter's previous command or a discrete filter state are each a function of this form. One channel exists, the deployment fraction f_b; its laws are memoryless and ignore c_{k-1}, so the mode currently changes only *how often* f_b is computed.

Three requirements follow from Equation 3.10.1:

1. **Fixed-step integration, with τ_s an exact integer multiple of the step.** An adaptive step cannot honour a hold boundary, and τ_s/h must be an integer so that every boundary falls on a step boundary. A non-integer ratio is rejected rather than rounded, which would silently change the sample rate.
2. **The hold is constant across every stage of a substep.** The right-hand side is then smooth inside a tick and the integrator's order is preserved; every discontinuity lands on a step boundary.
3. **The sample is taken at the tick's start**, $(t_k, \vec{y}(t_k))$: measure, compute, apply until the next tick, the causal order of a flight computer.

Two properties of a sampled run are not defects. It is a *different dynamical system* from the continuous one, not a more accurate solution of it; the meaningful comparison is "this actuator at this sample rate" against "an ideal continuous actuator". And it is substantially more expensive (small fixed steps for the whole flight), which is why it is opt-in: $f_s = 0$ is the default, and the mode is then structurally absent. It supplies the place to run a controller with memory, and nothing else: no such controller and no state-coupled turbulence model is written here.

Comparison (planned): continuous evaluation against the sampled mode at several f_s, on the demo rocket with the proportional air-brake law and fixed-step RK4; the table gives apogee and the final f_b, the plot $f_b(t)$ for each case. Generated by the simulator.

4 Other dynamics models

These models modify the equations of motion of section 2, or extend them by a degree of freedom, rather than supply a term of them. All are off by default.

4.1 Launch rail

On the launch rail the rocket can only slide along it: the rail buttons react every force component transverse to the rail and prevent rotation. This is a *kinematic constraint*, not a new force: the assembled acceleration is projected onto the rail, and the rotation is removed.

The rail direction is fixed and is the direction the nose initially points. With θ_{rail} the rail elevation, an input set consistently with the initial θ_r,

$$\hat{e}_{rail} = \cos\theta_{rail}\,\hat{x}_0 - \sin\theta_{rail}\,\hat{z}_0, \tag{4.1.1}$$

that is, $\hat{x}_r$ at $\theta_r = \theta_{rail}$, frozen. The distance travelled from a launch point at the origin is the projection of the position onto it,

$$s_{rail}(\vec{x}_r) = \vec{x}_r \cdot \hat{e}_{rail} = x\cos\theta_{rail} - z\sin\theta_{rail}. \tag{4.1.2}$$

The rocket is on the rail while $s_{rail} < L_{rail}$ (a state predicate), and there the equations of motion are replaced by

$$\vec{a}_r = \left[\left(\frac{\sum \vec{F}}{m} + \vec{a}^{\,2D}_{Coriolis}\right)\cdot \hat{e}_{rail}\right]\hat{e}_{rail} \tag{4.1.3}$$

$$\omega_r = 0, \qquad \varepsilon_r = 0 \tag{4.1.4}$$

with $\sum \vec{F}$ the sum of Equation 2.4.1 and $\vec{a}^{\,2D}_{Coriolis}$ the term of Equation 3.8.7 if enabled. The component of $\vec{a}_r$ normal to the rail is identically zero, and since $\vec{x}_r(0)$ lies on the rail line with $\vec{v}_r(0) = \vec{0}$, position and velocity stay on it with no correction step. In the integrator, Equation 4.1.4 reads $\mathrm{d}\theta_r/\mathrm{d}t = \mathrm{d}\omega_r/\mathrm{d}t = 0$, so θ_r and ω_r keep their initial values. The pitching moment is reacted by the rail rather than allowed to act.

Figure (planned): Diagram (TikZ): rail direction $\hat{e}_{rail}$, elevation θ_{rail}, length L_{rail} and the distance s_{rail} along it.

Exit and reverse motion. $L_{rail} = 0$ is the default and the off switch. Past rail exit ($s_{rail} \geq L_{rail}$) the free-flight equations apply again, with no transition term. Nothing holds the rocket on the pad before thrust exceeds weight. If the thrust along the rail is below the weight component, Equation 4.1.3 gives a negative $\vec{a}_r \cdot \hat{e}_{rail}$ and the rocket slides *backwards* down the rail ($s_{rail} < 0$) until thrust catches up. A real pad reacts weight until lift-off, and RASAero II corrected exactly this case [17]. The fix is to clamp $\vec{a}_r \cdot \hat{e}_{rail} \geq 0$ while $s_{rail} \leq 0$ and $\vec{v}_r \cdot \hat{e}_{rail} \leq 0$. (research: decision: adopt the clamp; it would change existing results for low thrust-to-weight starts)

Comparison (planned): no rail ($L_{rail} = 0$) against a rail of the demo launch site on the demo rocket; the table gives rail-exit speed, apogee and downrange. Generated by the simulator.

The key output is the rail-exit speed $|\vec{v}_r|$, which decides whether wind can tip the airframe before it can weathercock. There is no rail friction, flex or tip-off: the transverse and rotational freedoms are restored instantly at $s_{rail} = L_{rail}$.

4.2 Recovery

Once a canopy is deployed the attitude is frozen exactly as on the rail, $\omega_r = 0$, $\varepsilon_r = 0$, and only the translational equation changes. Deployment is instantaneous and total, and the canopy is modelled by its drag alone:

$$m\vec{a}_r = \vec{W} + \vec{T} + \vec{A}_f + \vec{R}_{ch} + m\,\vec{a}^{\,2D}_{Coriolis}, \tag{4.2.1}$$

with $\vec{R}_{ch}$ the canopy drag below and $\vec{a}^{\,2D}_{Coriolis}$ the term of Equation 3.8.7, present only when the Earth-rotation correction is enabled. The orientation is held at the pitch angle at deployment, and the pitching moment is not integrated. Freezing is chosen over an artificial damping term, which would need a time constant no source fixes. (sim: the Coriolis term is dropped in the code under canopy, although it is included on the rail and in free flight) (research: decision, no literature: in the code only $\mathrm{d}\theta_r/\mathrm{d}t$ and $\mathrm{d}\omega_r/\mathrm{d}t$ are set to zero, so the stored ω_r keeps its non-zero value at deployment and the reported pitch rate is meaningless under canopy. Reset it to zero?)

Deployment. A function of the state, not of a detected event: either a timer, $t \geq t_{dep}$, or apogee, taken as $v_z > 0$ (descending, z being positive down). The latter is reconstructed from the state because the right-hand side cannot consult an event not yet finalised, so a pre-apogee transient in v_z — possible on an unstable trajectory — would arm deployment early. The apogee trigger is ignored for the first 0.1 s, which covers a tail-first sag off the pad when the initial thrust is below the weight; later transients are not guarded against. These are the only two triggers; the predicate is not latched, but both are monotone on a nominal flight. While the rocket is on the rail the rail mode takes precedence. (sim: no altitude (or velocity/dynamic-pressure) deployment trigger exists; a main-chute-at-altitude trigger and a drogue-plus-main sequence are not implemented)

Force. The canopy drag is computed directly in $\mathcal{F}_0$, like weight and unlike the body-axis aerodynamic force:

$$\vec{R}_{ch} = -\tfrac{1}{2}\rho\, C_{D,ch} S_{ch}\, v_{rel}\, \vec{v}_{rel}, \tag{4.2.2}$$

with S_{ch} the canopy reference area and $C_{D,ch}$ its drag coefficient. Three properties are the whole content of the model:

- It is *natively inertial*, not routed through the body axial/normal decomposition. A round canopy is axisymmetric and its drag opposes the relative-wind *vector*, not the nose direction, which under canopy is frozen at whatever attitude deployment left. Routing it through $\hat{x}_r$ would make the descent rate depend on that attitude. $C_{D,ch}$ is therefore a drag coefficient on $\hat{v}_{rel}$, not the axial C_x, with its own reference area S_{ch}.
- It is *added to* the airframe's forces: weight, any residual thrust and the body-axis aerodynamic force all continue to act, evaluated at the frozen θ_r.
- It is written as $v_{rel}\,\vec{v}_{rel}$ rather than $\frac{1}{2}\rho v_{rel}^2 C_{D,ch} S_{ch}\,\hat{v}_{rel}$, so it has no 0/0 direction at $v_{rel} = 0$.

A run can be checked against the terminal descent speed, where $\vec{R}_{ch}$ balances weight, approached rather than attained since ρ varies:

$$v_{term} = \sqrt{\frac{2mg}{\rho\, C_{D,ch} S_{ch}}}. \tag{4.2.3}$$

$S_{ch} = 0$ is the default, as is "no recovery model"; either alone leaves this subsection inert. Inflation is instantaneous and total, with no opening transient, shock or reefing, and the frozen attitude replaces the pendulum swing under the risers.

Figure (planned): Plot (simulator output, a named demo scenario): altitude and vertical speed against time through deployment, with the terminal speed v_{term} of Equation 4.2.3 as a reference line.

Lumped alternative. Rocket and canopy can instead be one body with a combined drag area, $\vec{R}_{ch} = -\frac{1}{2}\rho\,(C_D S)_{sys}\, v_{rel}\, \vec{v}_{rel}$ with the airframe's own force dropped, where $(C_D S)_{sys}$ is a constant or a table against speed. For a steady descent this is sound and is the usual descent-rate and drift estimate: the drag areas of canopy and vehicle add, $v_{term} = \sqrt{2mg/\sum C_D S}$ [11], whereas OpenRocket and RocketPy take the drag from the recovery device alone and ignore the airframe [5, 12]. It cannot represent the inflation transient and opening shock, the pendulum motion, added mass or the riser dynamics, which need a drag area growing in time and a second body.

Neglected, with the place to look them up in Knacke [14]: the opening transient (drag area growing from zero over an inflation time, ch. 5.4.3–5.4.4), the opening-shock load on the airframe and shock cord (5.4.6), a reefed (5.6) or two-stage drogue and main sequence (ch. 7), the pendulum and coning motion of the body under the risers (5.3), and the added-mass effect of the air carried with the canopy. Knacke's rules come from large canopies ($Re \sim 10^7$) and under-predict the opening force and the inflation time of small ones [15]. (research: added-mass effect of the canopy: no source found)

Comparison (planned): the two deployment triggers (timer and apogee) on the demo rocket with a parachute; the table gives deployment altitude and speed, time of descent, landing distance and peak deceleration. Generated by the simulator.

4.3 Air brake

A switchable extra drag, rigidly mounted to the body, used to modulate drag during the unpowered coast. Its aerodynamics is part of the rocket's own: the deployment fraction $f_b \in [0, 1]$ is one more input to the coefficients of subsection 3.4, in one of two forms.

- **Own component.** The brake is a component b of the multi-component build-up (Equation 3.4.3) with its own reference area S_b and length c_b, coefficients $C_{x,b}, C_{z,b}, C_{m,b}$ and reference point pp_b, its force scaled by the exposed fraction, $S(f_b) = f_b S_b$:

$$\vec{A}_{f,b} = -f_b\, qS_b \left(C_{x,b}\hat{x}_r + C_{z,b}\hat{z}_r\right), \qquad \vec{A}_{m,b} = -f_b\, qS_b c_b\, C_{m,b}\hat{y}_r. \tag{4.3.1}$$

  It acts at pp_b, so Equation 3.4.3 also gives the pitching moment of its arm.

- **Whole-rocket tables with a deployment axis.** The coefficient tables take f_b as an extra axis, $(C_x, C_z, C_m, l_p) = \text{interp}(\alpha, M, f_b;\ \text{table})$, from CFD or wind-tunnel data with bilinear interpolation in M and f_b; this is common practice [13]. A brake can shift the centre of pressure through blanking of the fin flow (one design's static margin fell from 3.37 to 0.25 calibers at $M = 0.2$ [13]), so C_z and C_m should be tabulated against f_b too, not only C_x.

A flat plate with a constant drag coefficient is the usual simplification of the first form. The simplest case of the first form has only axial drag, $C_{z,b} = C_{m,b} = 0$ and $pp_b = p$, with $C_{x,b} = C_{D,b}$: one additive axial term

$$F_b = q\, C_{D,b}\, S_b\, f_b, \qquad \Delta C_x = \frac{C_{D,b} S_b}{S}\, f_b, \tag{4.3.2}$$

multiplied like C_x by the axial-flow factor Equation 3.4.4, so that it opposes the flow tail-first. $S_b = 0$ is the default. (sim: only this additive axial term is implemented; neither the own-component form nor the f_b table axis is) (research: no normal-force (C_N) data for deployed brakes found in the literature)

Two orthogonal choices set f_b: a *law*, which decides how far open the brake is, and a *trigger*, which decides from when it may open.

4.3.1 Deployment law

Three interchangeable variants:

- **None** (default): $f_b \equiv 0$.
- **Proportional**: P control on *predicted* apogee,

$$f_b = \text{clip}\Big(k_p \left(\hat{h}_a - h_{tgt}\right),\, 0,\, 1\Big), \qquad \hat{h}_a = h + \frac{v_z^2}{2g}, \tag{4.3.3}$$

  with h_{tgt} the commanded apogee and k_p the gain. $\hat{h}_a$ is the altitude the rocket would reach if drag vanished now, deliberately an *upper* bound that ignores all further drag, including the brake's own. A positive error (predicted overshoot) commands more brake. The law is gated to the unpowered-ascent window, returning 0 whenever $T(t) > 0$ or $v_z \geq 0$; that window belongs to the law and is not the trigger below.

- **Full**: $f_b = 1$ from the trigger instant onward and 0 before: a constant area, opened once. A sizing study solves against this variant: with f_b a step, a target apogee fixes either $C_{D,b}S_b$ or the deployment time, one unknown against one equation.

Every law is *memoryless*, re-evaluated from the current state at every right-hand-side call, since the solver evaluates at arbitrary, repeated and rejected times. A law with memory needs the sampled mode (subsection 3.10); none is written.

Comparison (planned): the three laws (none, proportional, full) on the demo rocket with an air brake; the table gives apogee, miss against h_{tgt} and time of maximum f_b, the plot the predicted apogee $\hat{h}_a$, f_b and altitude against time under the proportional law, with h_{tgt} marked. Generated by the simulator.

4.3.2 Deployment trigger

Which condition arms the brake, chosen independently of the law. Each variant is an *onset* latch: once true it stays armed.

- **After burnout** (default): $T(t) \leq 0$, true from burnout onward.
- **After time**: $t \geq t_{brake}$, with $t_{brake} = 0$ (default) meaning deployed from $t = 0$.
- **After apogee**: $v_z > 0$, the canopy's apogee predicate, with the same caveat: a pre-apogee transient in v_z would arm it early.

Only the **full** law reads the trigger; **none** ignores it and **proportional** uses its own window. No condition retracts an armed brake, and there is no deployment-rate limit between $f_b = 0$ and 1; a real actuator has both, and both are future work.

4.4 Roll and the reaction wheel

The rocket's own spin about its nose axis $\hat{x}_r$, and an internal flywheel that drives that spin by exchanging angular momentum with the airframe. This is the one model of this section that does not modify Equation 2.4.1 or Equation 2.4.3 at all: it *adds* a degree of freedom which, in the pitch plane, is exactly decoupled from both. The roll angle is γ_r about $+\hat{x}_r$ by the right-hand rule, the roll rate $\omega_x = \dot{\gamma}_r$ and the roll acceleration $\varepsilon_x = \dot{\omega}_x$, named after the pitch triple $\theta_r, \omega_r, \varepsilon_r$ of subsection 2.1. The vehicle's axial moment of inertia about $\hat{x}_r$ is I_x, taken with the wheel *included and locked*.

4.4.1 Why roll decouples exactly

For an axisymmetric body, $I_{yy} = I_{zz} = I$ with I the pitch inertia of subsection 3.3, and Euler's equation for the pitch channel reads

$$I\,\varepsilon_r = \left[\vec{gt} \times \vec{T} + \vec{gp} \times \vec{A}_f + \vec{A}_m\right] \cdot \hat{y}_r - (I_x - I)\,\omega_z\,\omega_x, \tag{4.4.1}$$

with ω_z the yaw rate about $\hat{z}_r$. Planar flight is $\omega_z \equiv 0$ (section 7), so the gyroscopic term vanishes *identically* — not to first order, and not for small ω_x, but for any roll rate whatever. Roll cannot drive pitch without a yaw rate to pair with. A wheel carrying axial momentum $h_w\hat{x}_r$ on a body pitching at $\omega_r\hat{y}_r$ likewise torques the airframe by

$$\vec{\tau}_{gyro} = -\,\vec{\omega} \times \vec{h} = -\,(\omega_r\hat{y}_r) \times (h_w\hat{x}_r) = +\,\omega_r h_w\,\hat{z}_r, \tag{4.4.2}$$

which is a *yaw* torque: no $\hat{y}_r$ component, so it cannot enter Equation 4.4.1, and no $\hat{x}_r$ component, so it cannot enter the roll equation either. Finally, C_x, C_z, C_m and $C_{m,q}$ of subsection 3.4 are functions of (α, M) alone and take no roll angle. So the three channels — translation, pitch, roll — share only the flow state, and the roll equations below can be integrated alongside the others with no term crossing between them.

What the decoupling *hides* is the yaw that both Equation 4.4.1 and Equation 4.4.2 produce, which a 6DOF model would carry and this one cannot represent at all — exactly as the out-of-plane wind component is computed and discarded. Two further effects are dropped with it: for a *finned* airframe C_z and C_m do depend weakly on γ_r (a four-fin body at incidence varies with a 90° period, through fin–body interference and the fins' effective dihedral relative to the α plane), and at high spin there is roll lock-in with the pitch natural frequency and a Magnus side force. (research: roll-orientation dependence of C_z, C_m for a four-fin airframe: needs a $C_z(\alpha, M, \gamma_r)$ table axis, for which no data source is identified)

4.4.2 Equations of motion

The vehicle is two bodies sharing one axis: the airframe, and a balanced wheel spinning about $\hat{x}_r$ with its own axial inertia I_w and absolute rate ω_w. Only two torques act about that axis — the external aerodynamic moment L_x on the airframe, and the motor torque τ_w, which by Newton's third law is $+\tau_w$ on the wheel and $-\tau_w$ on the airframe:

$$\dot{\gamma}_r = \omega_x \tag{4.4.3}$$

$$(I_x - I_w)\,\varepsilon_x = L_x - \tau_w \tag{4.4.4}$$

$$I_w\,\dot{\omega}_w = \tau_w \tag{4.4.5}$$

This is the form to integrate, and it is the reason the wheel's *absolute* rate is carried rather than its relative one: Equation 4.4.5 is then a standalone scalar equation with no ε_x on its right-hand side. The rate a motor encoder reads is the relative $\omega_{w/r} = \omega_w - \omega_x$, and the wheel's stored momentum is $h_w = I_w\,\omega_{w/r}$.

Summing Equation 4.4.4 and Equation 4.4.5 gives the same content as a momentum balance, which is how the device is best read:

$$h_x = I_x\,\omega_x + h_w, \qquad \dot{h}_x = L_x. \tag{4.4.6}$$

With no external moment and the vehicle starting at rest, $h_x \equiv 0$ and

$$\omega_x = -\,\frac{h_w}{I_x}\,: \tag{4.4.7}$$

spin the wheel up to momentum h_w and the airframe counter-rotates, using no external torque at all. That is the whole purpose of a reaction wheel, and Equation 4.4.7 is the check a run is verified against.

4.4.3 Authority and saturation

The device is fictitious unless both of its limits are carried:

$$|\tau_w| \le \tau_{max}, \qquad |\omega_{w/r}| \le \omega_{w,max}. \tag{4.4.8}$$

The first is the motor's stall torque. The second is the one that matters: on reaching it the wheel *saturates*, Equation 4.4.7 caps the roll rate it can hold at $I_w\omega_{w,max}/I_x$, and all authority in that direction is gone until an *external* moment desaturates it. The speed limit is one-sided in the model — at saturation the wheel may still be braked back, only accelerated further is forbidden — since a two-sided clamp would lock it at its limit for the rest of the flight. Both limits make τ_w discontinuous in the state, exactly as the air brake's clip does in Equation 4.3.3, and no event is located at the boundary. (research: I_x of the airframe, and I_w, τ_{max}, $\omega_{w,max}$ of a wheel sized for it: no component is chosen, so no numbers exist)

4.4.4 Aerodynamic roll moment

Roll is damped, and can be driven, by the fins. In the convention of Appendix C, with the non-dimensional roll rate formed like the pitch rate $\hat{\omega}$ of Equation 3.4.11,

$$\vec{A}_{m,x} = -qSc\,(C_l + C_{l,p}\,\hat{\omega}_x)\,\hat{x}_r, \qquad \hat{\omega}_x = \frac{\omega_x c}{2v_{rel}}, \tag{4.4.9}$$

so that $L_x = \vec{A}_{m,x}\cdot\hat{x}_r$, with C_l the static roll-moment coefficient of a canted fin set and $C_{l,p} > 0$ damping, as $C_{m,q} > 0$ damps in pitch. The damping term is evaluated as $\frac{1}{4}\rho v_{rel}\omega_x c^2 C_{l,p}$, algebraically identical but finite at $v_{rel} = 0$, which the literal quotient is not. Both coefficients are zero by default, which makes $L_x \equiv 0$ and leaves Equation 4.4.7 — pure momentum exchange — as the model's default behaviour. A motor also produces a roll torque about $\hat{x}_r$ from its internal vortex flow and nozzle ablation (subsection 3.7); it belongs in L_x, and no prediction method for it is known. (research: C_l and $C_{l,p}$ for this airframe: the fin cant is not chosen and no roll data exists; Barrowman-style build-up or CFD would supply both)

4.4.5 Control law and the constrained phases

The commanded torque is a memoryless function of the state, re-evaluated at every right-hand-side call for the reason given in subsubsection 4.3.1. Three variants:

- **None** (default): $\tau_w \equiv 0$. The wheel is present but never commanded, so the roll that results is whatever the initial condition and L_x produce.
- **Rate null**: $\tau_w = k_d\,\omega_x$, which by Equation 4.4.4 opposes the roll rate — the despin law.
- **Angle hold**: $\tau_w = k_p\,(\gamma_r - \gamma_{cmd}) + k_d\,\omega_x$, a PD hold on a commanded roll angle. There is no integral term: one needs the sampled mode (subsection 3.10) to carry its accumulator, and none is written.

Both gains are positive, and both are clipped by Equation 4.4.8.

On the launch rail (subsection 4.1) the buttons running in the rail prevent roll as they prevent pitch, so $\omega_x = 0$ and $\varepsilon_x = 0$ there, alongside Equation 4.1.4; under a canopy (subsection 4.2) roll is frozen the same way, inheriting that section's choice rather than deserving it — a real rocket under a canopy spins freely. In both phases Equation 4.4.5 keeps running: the wheel is internal, its reaction torque is taken by the rail buttons or the risers instead of by the airframe's roll freedom, and a wheel spun up on the rail therefore reaches rail exit with real stored momentum. (sim: the sampled mode carries no roll channel, so a roll law with memory (integral term, rate limiting on τ_w, or a measured rather than true ω_x) is not expressible)

Figure (planned): Plot (simulator output, a named demo scenario): roll angle γ_r, roll rate ω_x and relative wheel rate $\omega_{w/r}$ against time under the angle-hold law, with the saturation level $\omega_{w,max}$ of Equation 4.4.8 as a reference line.

Comparison (planned): the three laws (none, rate null, angle hold) on the demo rocket given an initial roll rate; the table gives the roll angle at apogee, the peak relative wheel rate against $\omega_{w,max}$, and whether the wheel saturated. Generated by the simulator.

5 Rocket 3D

This section lifts section 2 out of the pitch plane. The frames, the points, the forces and the translational equation of motion all survive unchanged in *form* — only the number of components they carry grows. One thing is genuinely new and is the whole subject here: rotation about three axes at once, which needs an attitude representation that does not break pointing straight up, a full moment-of-inertia tensor instead of the single pitch inertia I, and Euler's rotational equations in place of the scalar Equation 2.4.3.

5.1 What 3D changes, and what it does not

subsection 2.1 already defines a right-handed body triad $\hat{x}_r, \hat{y}_r, \hat{z}_r$; the planar model simply never rotated about two of those axes. Nothing in that definition needs amending. Likewise the points A, g, t, p and the arms $\vec{gt}, \vec{gp}$ of Equation 2.3.2 are points and body-frame distances, not planar constructions, and the forces $\vec{W}, \vec{T}, \vec{A}_f$ generalise component-wise. Translation, Equation 2.4.1, is therefore unchanged as written; only the rotation matrix taking body components to ground components is now a full 3×3 attitude matrix rather than the two lines of Equation 2.1.1.

So three things change, and they are the contents of this section:

- **Attitude.** A single pitch angle θ_r cannot describe a general orientation. subsection 5.2 replaces it with a quaternion.
- **Inertia.** The scalar I of subsection 3.3 is one component of a symmetric tensor with six independent entries. subsection 5.3 and subsection 5.4 set it up, including which tensors are physically possible.
- **Rotational dynamics.** Equation 2.4.3 is a scalar statement about one axis. A body rotating about three axes acquires the gyroscopic term $\vec{\omega} \times \mathbf{I}\vec{\omega}$, which has no planar analogue at all — a single rotation axis has nothing to precess around. subsection 5.5.

The angular velocity is now a vector, $\vec{\omega} = \omega_x \hat{x}_r + \omega_y \hat{y}_r + \omega_z \hat{z}_r$: ω_x the roll rate of subsection 4.4, ω_y the pitch rate written ω_r throughout section 2, and ω_z the yaw rate that the planar model sets identically to zero (section 7). The identification $\omega_y \equiv \omega_r$ is exact, not a convention chosen here: $\hat{y}_r$ is the pitch-plane normal in subsection 2.1 and θ_r is positive nose-up about $+\hat{y}_r$.

What 3D does *not* buy is better numbers for a nominal flight. For a symmetric vehicle flown near-vertically in a uniform wind, the 3D answer is the 2D answer: apogee, maximum dynamic pressure, burnout velocity and static margin are all essentially unchanged. 3D is about *asymmetric* flight — roll, out-of-plane wind response, sideslip, and the mass asymmetry this section's full tensor exists to carry.

5.2 Attitude: the quaternion

A three-angle sequence is the obvious generalisation of θ_r and the wrong one here. The standard 3–2–1 (yaw–pitch–roll) sequence is singular at a pitch of $\pm 90°$, which is exactly where a vertically launched rocket sits for its entire boost phase: the attitude is perfectly well defined there, but the angles that describe it are not, and their rates diverge.

Let $q = (q_0, q_1, q_2, q_3)$ be the unit quaternion representing the rotation from $\mathcal{F}_0$ to $\mathcal{F}_r$, scalar part first. Its kinematics are the standard result (Wertz [25], or any flight-dynamics text):

$$\dot{q} = \tfrac{1}{2}\, q \otimes (0,\ \vec{\omega})\,, \tag{5.2.1}$$

with $\otimes$ the Hamilton product and the body rate $\vec{\omega}$ embedded as a pure quaternion. In components, which is the form actually integrated:

$$\begin{aligned} \dot{q}_0 &= -\tfrac{1}{2}\left(q_1\omega_x + q_2\omega_y + q_3\omega_z\right), & \dot{q}_1 &= \tfrac{1}{2}\left(q_0\omega_x + q_2\omega_z - q_3\omega_y\right), \\ \dot{q}_2 &= \tfrac{1}{2}\left(q_0\omega_y + q_3\omega_x - q_1\omega_z\right), & \dot{q}_3 &= \tfrac{1}{2}\left(q_0\omega_z + q_1\omega_y - q_2\omega_x\right). \end{aligned} \tag{5.2.2}$$

There is no singularity anywhere in Equation 5.2.2, and the price is one extra state and one constraint, $\|q\| = 1$.

The planar case falls out exactly. A pure pitch rotation is $q = \left(\cos\tfrac{\theta_r}{2},\ 0,\ \sin\tfrac{\theta_r}{2},\ 0\right)$. Substituting it with $\omega_x = \omega_z = 0$ into Equation 5.2.2 kills $\dot{q}_0$'s q_1, q_3 terms and both $\dot{q}_1$ and $\dot{q}_3$ identically, leaving

$$\dot{q}_0 = -\tfrac{1}{2}\sin\tfrac{\theta_r}{2}\,\omega_y, \qquad \dot{q}_2 = \tfrac{1}{2}\cos\tfrac{\theta_r}{2}\,\omega_y, \tag{5.2.3}$$

which is the half-angle form of $\dot{\theta}_r = \omega_r$ — the planar kinematics of Equation 3.9.1, recovered without approximation. The attitude matrix built from this q likewise reproduces Equation 2.1.1 exactly.

Norm drift. Equation 5.2.1 preserves $\|q\| = 1$ in exact arithmetic only; in floating point the norm wanders. The constraint is therefore restored by renormalising q, either at every accepted integrator step or as a solver event — an integration concern with no physics in it, and the drift between restorations is a measured quantity rather than an assumed-small one. (sim: renormalisation is not yet applied anywhere: the right-hand side deliberately does not project q, because a projection applied inside the right-hand side changes the vector field the error controller sees. It belongs at step boundaries, which requires the 3D integration harness that does not exist yet (subsection 5.7))

5.3 The inertia tensor

In the pitch plane, all of rotational inertia is the single number $I(t)$ of Equation 3.3.7: the moment of inertia about $\hat{y}_r$ through g. A body rotating about three axes needs the full tensor, and this document takes it in its *general* symmetric form — six independent components, with no assumption that the body axes are principal axes and no assumption that the two transverse moments are equal.

That is a deliberate widening of what subsubsection 4.4.1 assumes. A rocket built perfectly as a body of revolution about $\hat{x}_r$ has $I_{yy} = I_{zz}$ and vanishing products of inertia, by symmetry; the general tensor exists precisely to carry the *departure* from that ideal — a heavier avionics bay on one side, an off-axis payload, a bent airframe — which is real on any built vehicle and which the symmetry argument cannot represent at all.

Definitions, and the sign that matters. Taking moments about g, with body-frame coordinates (x, y, z) measured from g:

$$\begin{aligned} I_{xx} &= \int (y^2 + z^2)\,\mathrm{d}m, & I_{yy} &= \int (x^2 + z^2)\,\mathrm{d}m, & I_{zz} &= \int (x^2 + y^2)\,\mathrm{d}m, \\ I_{xy} &= \int xy\,\mathrm{d}m, & I_{xz} &= \int xz\,\mathrm{d}m, & I_{yz} &= \int yz\,\mathrm{d}m. \end{aligned} \tag{5.3.1}$$

The first three are the moments of inertia, the last three the *products* of inertia. The angular momentum about g is $\vec{H} = \int \vec{r} \times (\vec{\omega} \times \vec{r})\,\mathrm{d}m$; expanding with $\vec{r} \times (\vec{\omega} \times \vec{r}) = \vec{\omega}\,(\vec{r} \cdot \vec{r}) - \vec{r}\,(\vec{r} \cdot \vec{\omega})$ gives, for the first component,

$$H_x = \omega_x \int (y^2 + z^2)\,\mathrm{d}m - \omega_y \int xy\,\mathrm{d}m - \omega_z \int xz\,\mathrm{d}m,$$

and cyclically for H_y, H_z. So $\vec{H} = \mathbf{I}\vec{\omega}$ with

$$\mathbf{I} = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{xy} & I_{yy} & -I_{yz} \\ -I_{xz} & -I_{yz} & I_{zz} \end{bmatrix}. \tag{5.3.2}$$

The minus signs are a consequence of Equation 5.3.1, not a choice: they appear because I_{xy} is defined as the integral $\int xy\,\mathrm{d}m$ rather than as the matrix entry itself. Both conventions are in use — the one above is standard in aerospace texts (Hibbeler [26], Stevens and Lewis [27]) and is what a CAD mass-properties report calls "products of inertia"; much of the physics literature instead defines the tensor components directly as $-\int x_i x_j\,\mathrm{d}m$ for $i \neq j$ and so carries no extra sign. *Numbers transcribed from a source using the other convention will have the wrong sign on every off-diagonal term.* The convention of Equation 5.3.1 is the one used throughout this document and is recorded in Appendix C.

Relation to the planar I. I_{yy} is the quantity subsubsection 3.3.3 calls I, and I_{xx} is the axial inertia subsection 4.4 calls I_x — the same physical numbers, reached from a different direction. Two differences are worth stating rather than leaving to be discovered:

- Equation 3.3.7 gives $I(t)$ as a function of time, because the propellant burns and g travels along the axis. The tensor of Equation 5.3.2 is taken *constant*. Where the mass model is active the two therefore agree only at the instant the tensor is evaluated.
- I_{xx} is *not* shifted by the parallel-axis theorem as g travels, because axial inertia is built from perpendicular distances to the axis, which an axial shift of the reference point does not change. It does, however, fall with the propellant mass; it is independent of $l_g(t)$, not of $m(t)$.

(sim: the simulator carries the tensor as six constant inputs, separate from the time-varying $I(t)$ of Equation 3.3.7, because the 3D path has no variable-mass model yet. A time-varying tensor adds the term $\dot{\mathbf{I}}\vec{\omega}$ to Equation 5.5.1, and nothing in this document yet derives $\dot{\mathbf{I}}$ for a burning grain) (research: no measured mass properties exist for the demo airframe: neither I_{xx} nor the three products of inertia. The same airframe-geometry gap already named in section 8 for Barrowman and $C_{m,q}$)

5.4 Which tensors are physically possible

Not every symmetric matrix is an inertia tensor, and the constraints are sharp enough to be worth checking against before a tensor is trusted. Both follow from Equation 5.3.1 in one line each.

Positive definiteness. For any $\vec{\omega}$,

$$\vec{\omega} \cdot \mathbf{I}\vec{\omega} = \int \|\vec{\omega} \times \vec{r}\|^2 \, \mathrm{d}m \;\geq\; 0, \tag{5.4.1}$$

which is twice the rotational kinetic energy. So $\mathbf{I}$ is always positive *semi*-definite, and positive *definite* unless every mass element lies on one line through g. A real vehicle is not a line, so all three principal moments — the eigenvalues $\lambda_1 \leq \lambda_2 \leq \lambda_3$ of $\mathbf{I}$ — are strictly positive.

Triangle inequalities. In any orthonormal body frame,

$$I_{xx} + I_{yy} - I_{zz} = \int \left[\left(y^2 + z^2\right) + \left(x^2 + z^2\right) - \left(x^2 + y^2\right) \right] \mathrm{d}m = 2 \int z^2 \, \mathrm{d}m \;\geq\; 0, \tag{5.4.2}$$

and cyclically; on the principal axes this reads $\lambda_1 + \lambda_2 \geq \lambda_3$, which is the binding one of the three. Positive definiteness alone is not enough: $\mathrm{diag}(1, 1, 10)$ is positive definite and corresponds to no body whatever.

Together they are sufficient. Given $\lambda_i > 0$ satisfying the triangle inequalities strictly, the uniform solid ellipsoid of mass m with semi-axes

$$a^2 = \frac{5}{2m}\left(\lambda_2 + \lambda_3 - \lambda_1\right), \quad b^2 = \frac{5}{2m}\left(\lambda_1 + \lambda_3 - \lambda_2\right), \quad c^2 = \frac{5}{2m}\left(\lambda_1 + \lambda_2 - \lambda_3\right) \tag{5.4.3}$$

has exactly those principal moments. So Equation 5.4.1 and Equation 5.4.2 are necessary *and* sufficient for a tensor to be realisable by some mass distribution: a check against them rejects only the impossible, with no conservatism and no tuning.

The consequence for a slender vehicle. Combining the two triangle inequalities that contain I_{xx},

$$I_{xx} + I_{yy} - I_{zz} \geq 0 \;\text{ and }\; I_{xx} + I_{zz} - I_{yy} \geq 0 \qquad \Longleftrightarrow \qquad |I_{yy} - I_{zz}| \;\leq\; I_{xx}. \tag{5.4.4}$$

The two transverse moments of *any* body differ by at most its axial moment. For a rocket that is a severe restriction, because I_{xx} is two to three orders of magnitude below I_{yy}: with the demo airframe's figures ($m \approx 1.5\,\mathrm{kg}$, $I_{yy} \approx 0.15\,\mathrm{kg\,m^2}$, body radius 27 mm, so $I_{xx} \approx 5\times10^{-4} \ldots 1.1\times10^{-3}\,\mathrm{kg\,m^2}$ between a solid cylinder and a thin shell) the two transverse moments may differ by no more than about half a percent of their own magnitude. This is not a modelling limitation but the geometry of a slender body: for an axisymmetric mass distribution $\int y^2 \mathrm{d}m = \int z^2 \mathrm{d}m$, so the axial moment $I_{xx} = 2\int z^2 \mathrm{d}m$ *is* the slack in Equation 5.4.2 — and a slender body has very little of it to spend. The freedom the general tensor really adds is therefore the *off-diagonal* one — body axes that are not principal axes — which carries no comparably narrow bound.

5.5 Rotational equations of motion

Newton's second law for rotation is $\mathrm{d}\vec{H}/\mathrm{d}t = \vec{M}$ in the inertial frame. $\vec{H}$ is carried in body components, and the body frame itself rotates at $\vec{\omega}$, so by the transport theorem $\mathrm{d}\vec{H}/\mathrm{d}t\big|_{\mathcal{F}_0} = \dot{\vec{H}}\big|_{\mathcal{F}_r} + \vec{\omega} \times \vec{H}$. For a rigid body of fixed mass distribution $\mathbf{I}$ is constant in $\mathcal{F}_r$, so $\dot{\vec{H}}\big|_{\mathcal{F}_r} = \mathbf{I}\dot{\vec{\omega}}$, giving Euler's rotational equation in its general form:

$$\mathbf{I}\dot{\vec{\omega}} + \vec{\omega} \times (\mathbf{I}\vec{\omega}) = \vec{M}. \tag{5.5.1}$$

This is the 3D replacement for Equation 2.4.3. Two conditions are built into it and are easily violated by accident: moments and inertia are both taken about g (or another body-fixed point), and $\mathbf{I}$ is constant — a time-varying tensor adds $\dot{\mathbf{I}}\vec{\omega}$ on the left.

Written out for integration, with $\vec{H} = \mathbf{I}\vec{\omega}$ from Equation 5.3.2:

$$\begin{aligned} H_x &= \;\; I_{xx}\omega_x - I_{xy}\omega_y - I_{xz}\omega_z, \\ H_y &= -I_{xy}\omega_x + I_{yy}\omega_y - I_{yz}\omega_z, \\ H_z &= -I_{xz}\omega_x - I_{yz}\omega_y + I_{zz}\omega_z, \end{aligned} \tag{5.5.2}$$

the rotational equation becomes a 3×3 linear system in $\dot{\vec{\omega}}$:

$$\mathbf{I}\dot{\vec{\omega}} = \vec{M} - \begin{bmatrix} \omega_y H_z - \omega_z H_y \\ \omega_z H_x - \omega_x H_z \\ \omega_x H_y - \omega_y H_x \end{bmatrix}. \tag{5.5.3}$$

This is the form evaluated once per right-hand-side call. It is well conditioned for any admissible tensor (subsection 5.4), though a slender airframe's condition number is of order $I_{yy}/I_{xx} \approx 300$, which is reason enough to solve the system rather than form $\mathbf{I}^{-1}$ explicitly.

Reduction to principal axes. With the products of inertia zero, $\vec{H} = (I_{xx}\omega_x,\, I_{yy}\omega_y,\, I_{zz}\omega_z)$ and Equation 5.5.3 separates into the classical Euler equations:

$$\begin{aligned} I_{xx}\dot{\omega}_x + (I_{zz} - I_{yy})\,\omega_y\omega_z &= M_x, \\ I_{yy}\dot{\omega}_y + (I_{xx} - I_{zz})\,\omega_z\omega_x &= M_y, \\ I_{zz}\dot{\omega}_z + (I_{yy} - I_{xx})\,\omega_x\omega_y &= M_z. \end{aligned} \tag{5.5.4}$$

Reduction to the axisymmetric body. Setting $I_{yy} = I_{zz} = I$ and $I_{xx} = I_x$, the notation of subsection 3.3 and subsection 4.4:

$$\begin{aligned} I_x\dot{\omega}_x &= M_x, \\ I\dot{\omega}_y &= M_y - (I_x - I)\,\omega_z\omega_x, \\ I\dot{\omega}_z &= M_z - (I - I_x)\,\omega_x\omega_y. \end{aligned} \tag{5.5.5}$$

The middle line is Equation 4.4.1 exactly, which is the cross-check that the signs of Equation 5.5.3 are right. Three features of Equation 5.5.5 are worth naming:

- **The roll axis carries no gyroscopic term at all** — the coefficient would be $I - I = 0$. This is why the decoupled roll equation Equation 4.4.4 of subsection 4.4 is not an approximation but literally the roll component of the full 3D system, and why it carries over to 3D unchanged.
- **With $\omega_x = 0$ the transverse axes decouple** into two independent copies of the planar Equation 2.4.3. A non-rolling rocket has no pitch–yaw coupling.
- **With $\omega_x = \omega_z = 0$ the pitch line becomes $I\dot{\omega}_y = M_y$**, which is Equation 2.4.3 itself. The planar model is the exact restriction of this one, not a separate theory.

What this recovers that the planar model discarded. subsubsection 4.4.1 shows the roll–pitch gyroscopic torque and the reaction wheel's own gyroscopic torque (Equation 4.4.2) are both pure *yaw* torques, and that a planar model can only compute them and throw them away. In Equation 5.5.1 they are simply there: the first as the M_z-row coupling of Equation 5.5.5, the second once the wheel's momentum is added to $\vec{H}$. (sim: the reaction wheel is not yet part of the 3D momentum $\vec{H}$: the 3D tensor describes the airframe with the wheel included and locked, as I_x already does in subsection 4.4. A spinning wheel in 3D adds $\vec{h}_w$ to $\vec{H}$ and so a term $\vec{\omega} \times \vec{h}_w$ to Equation 5.5.1)

5.6 The moment vector

Equation 2.4.3 takes the bracket of thrust, aerodynamic-force and aerodynamic-moment contributions and projects it onto $\hat{y}_r$. The 3D moment is that same bracket *without* the projection:

$$\vec{M} = \vec{gt} \times \vec{T} + \vec{gp} \times \vec{A}_f + \vec{A}_m, \tag{5.6.1}$$

with the arms of Equation 2.3.2 read as 3-vectors and $\vec{A}_m$ now carrying roll and yaw components alongside the pitching one of Equation 2.2.3. Weight still contributes nothing, since it acts at g.

The three components are not equally well supplied. M_y is the planar pitching moment and is fully modelled (subsection 3.4). M_z, the yaw moment, needs the aerodynamic response to sideslip — the same coefficients as pitch for a body of revolution, driven by the out-of-plane component of the relative wind instead of the in-plane one. M_x, the roll moment, needs fin cant and roll damping, which is Equation 4.4.9 of subsection 4.4. (sim: only M_y is wired: the simulator currently evaluates Equation 5.6.1 as $\vec{M} = (0,\, M_y,\, 0)$ with M_y taken from the planar chain, so a 3D run reproduces the planar pitch history and nothing more. M_x and M_z arrive with the 3D aerodynamic model) (research: the 3D aerodynamic moment needs the total angle of attack and the sideslip angle in place of the planar α, a roll-moment coefficient from fin cant, and roll damping. The body-of-revolution

coefficient set of subsection 3.4 is already written in terms of a total α, so the extension is a matter of resolving the relative wind in 3D rather than new coefficient data — except for the roll pair, for which no data exists)

A second gap sits in Equation 2.3.2 rather than in the coefficients: the arms $\vec{gt}, \vec{gp}$ carry the radial offsets r_g, r_t along $-\hat{z}_r$ only, because in the pitch plane there is nowhere else for them to point. In 3D each becomes a two-component radial offset, and whether the arms follow $l_g(t)$ as the centre of gravity travels — an open question already in the planar model — becomes the same question twice over, once for pitch and once for yaw. (research: radial offsets as 3D vectors, and whether the moment arms track $l_g(t)$: one error term in the planar model, the same term in both transverse channels in 3D)

5.7 State vector, and what remains

The seven-component planar state of Equation 3.9.2 becomes thirteen:

$$\vec{y} = (\vec{x}_r,\ \vec{v}_r,\ q,\ \vec{\omega}) = (x, y, z,\ v_x, v_y, v_z,\ q_0, q_1, q_2, q_3,\ \omega_x, \omega_y, \omega_z)\,, \tag{5.7.1}$$

with z and v_z positive *down* as before, q the attitude of subsection 5.2 and $\vec{\omega}$ the body rate. Position and velocity are resolved in $\mathcal{F}_0$; $\vec{\omega}$ is resolved in $\mathcal{F}_r$, which is what Equation 5.5.1 is written in. The derivatives are $\dot{\vec{x}}_r = \vec{v}_r$ and $\dot{\vec{v}}_r = \vec{a}_r$ from Equation 2.4.1, $\dot{q}$ from Equation 5.2.2, and $\dot{\vec{\omega}}$ from Equation 5.5.3.

Mass is absent from Equation 5.7.1, unlike Equation 3.9.2: the 3D formulation above is written for a rigid body of fixed mass distribution, consistent with the constant tensor of subsection 5.3. (sim: the 3D state is integrated by the same schemes as the planar one (Equation 3.9.4), but none of the flight-phase machinery is generalised: the rail constraint (subsection 4.1), the canopy phase (subsection 4.2), and the ground, apogee and burnout event detection are all written against Equation 3.9.2. A 3D scenario therefore cannot yet be flown end to end — the rotational dynamics of this section are exercised over a fixed time span only. In particular the attitude freezing that subsection 4.1 and subsection 4.2 apply to θ_r, ω_r has no 3D counterpart yet, and must freeze $\dot{q}$ and $\dot{\vec{\omega}}$ together when it arrives)

The validation this formulation supports today, in increasing strength: the planar restriction of Equation 5.5.5 reproduces the pitch history of Equation 2.4.3 with the out-of-plane components staying identically zero; torque-free motion of an asymmetric tensor conserves $\|\mathbf{I}\vec{\omega}\|$ and the rotational kinetic energy of Equation 5.4.1; and, strongest, the angular momentum $\vec{H}$ resolved back into $\mathcal{F}_0$ through the attitude of subsection 5.2 is constant under zero moment, which ties Equation 5.2.2 and Equation 5.5.1 together and fails on any frame or convention mismatch between them.

6 Analytical analysis

This section works through the pitching moment — thrust on or off, thrust axis aligned or misaligned — and the static and dynamic stability results that follow. An acceleration component such as $\vec{a}_r \cdot \hat{x}_r$ is the acceleration resolved along the body axes at the current instant, not a derivative taken inside the rotating frame $\mathcal{F}_r$.

Two inputs are zero by default and reduce everything below to its simplest form: the thrust angle θ_t and the gravity-direction rotation $\varphi = x/R_e$, which stays zero if the local vertical is frozen at the launch point.

6.1 Forces

Aerodynamic force and thrust are already body-axis quantities, so only weight has to be resolved. Weight acts along the local vertical $\hat{z}_g(\varphi) = -\sin\varphi\,\hat{x}_0 + \cos\varphi\,\hat{z}_0$, so with Equation 2.1.1

$$\vec{W} \cdot \hat{x}_r = -mg\sin(\theta_r + \varphi), \qquad \vec{W} \cdot \hat{z}_r = +mg\cos(\theta_r + \varphi). \tag{6.1.1}$$

The downrange rotation of gravity thus enters only as a shift of the pitch angle, $\theta_r \to \theta_r + \varphi$: the pitch above the *local* horizon. Adding thrust and the aerodynamic force,

$$m\,\vec{a}_r \cdot \hat{x}_r = -mg\sin(\theta_r + \varphi) + T\cos\theta_t - qSC_x \tag{6.1.2}$$

$$m\,\vec{a}_r \cdot \hat{z}_r = mg\cos(\theta_r + \varphi) - T\sin\theta_t - qSC_z \tag{6.1.3}$$

Sign checks, with $\varphi = \theta_t = 0$. Nose straight up ($\theta_r = 90°$): weight is purely axial, $m\,\vec{a}_r \cdot \hat{x}_r = -mg + T - qSC_x$. Nose horizontal ($\theta_r = 0$): weight is purely normal, $m\,\vec{a}_r \cdot \hat{z}_r = mg - qSC_z$. For φ, flying level at downrange $x > 0$ gives $m\,\vec{a}_r \cdot \hat{x}_r = -mg\sin\varphi < 0$: the Earth's centre lies below and behind the rocket, so the pull is tilted against the flight direction. At $x = 10\,\text{km}$ this is $g\varphi \approx 1.5 \times 10^{-2}\,\text{m/s}^2$.

6.2 Moment

Substituting the arms of Equation 2.3.2 into Equation 2.4.3, with $\hat{x}_r \times \hat{z}_r = -\hat{y}_r$ and the thrust vector of Equation 3.2.4:

$$I\varepsilon_r = (r_g - r_t)T\cos\theta_t + (l_g - l_t)T\sin\theta_t - r_g\,qSC_x + (l_g - l_p)\,qSC_z - qSc\,C_m \tag{6.2.1}$$

Thrust acts at t, displaced from the CoG by $\vec{gt} = (l_g - l_t)\hat{x}_r + (r_g - r_t)\hat{z}_r$: the radial offset gives an arm to the axial thrust component $T\cos\theta_t$, the axial offset an arm to the normal component $T\sin\theta_t$. The aerodynamic force acts at p: the axial arm $l_g - l_p$ carries qSC_z and the radial arm r_g carries qSC_x. Weight does not appear, since $\vec{gg} \times \vec{W} = 0$ for every orientation of $\vec{W}$. Rotating the local vertical is therefore a purely translational correction, like the Coriolis term.

For on-axis thrust, $\vec{T} = T\hat{x}_r$, sliding the application point along the axis changes nothing: the force is parallel to the displacement. Only r_t survives. Once $\theta_t \neq 0$ the normal component does care how far along the axis it acts, hence the second term.

Equation 6.2.1 specialises four ways; *aligned* means $r_g = r_t = \theta_t = 0$.

	aligned ($\theta_t = 0$)	**misaligned** ($\theta_t \neq 0$)
no thrust ($T = 0$)	Equation 6.2.2	identical — $T = 0$ removes θ_t
with thrust ($T > 0$)	Equation 6.2.3	Equation 6.2.4

6.2.1 No thrust: coast and descent

Setting $T = 0$ leaves the airframe's purely aerodynamic behaviour, independent of the motor:

$$I\varepsilon_r = -r_g\,qSC_x + (l_g - l_p)\,qSC_z - qSc\,C_m. \tag{6.2.2}$$

Since $T = 0$ removes both thrust terms, misalignment is irrelevant here, and the static-stability result below, built on this case, needs no revision when it is switched on.

6.2.2 With thrust, aligned

With $r_g = r_t = 0$ and $\theta_t = 0$,

$$I\varepsilon_r = (l_g - l_p)\,qSC_z - qSc\,C_m. \tag{6.2.3}$$

Thrust has dropped out exactly: an aligned rocket's attitude dynamics are the same during boost as during coast, however large T is.

6.2.3 With thrust, misaligned

With the nozzle canted by a small fixed θ_t and the geometry otherwise on-axis,

$$I\varepsilon_r = \underbrace{(l_g - l_t)T\sin\theta_t}_{M_{\theta_t}} + (l_g - l_p)\,qSC_z - qSc\,C_m. \tag{6.2.4}$$

Unlike every aerodynamic term, M_{θ_t} survives $q = 0$ and $\alpha = 0$: while the motor burns, the airframe carries a disturbance torque proportional to the thrust. Its sign: $l_t > l_g$ for any real rocket, so $l_g - l_t < 0$. A positive θ_t pushes the tail towards $-\hat{z}_r$ and pitches the nose *down*, $M_{\theta_t} < 0$; a negative θ_t pitches it up.

Since $\partial M_{\theta_t}/\partial\alpha = 0$, the stability derivative of Equation 6.3.2 is untouched: misalignment shifts the equilibrium without changing whether it is stable. Setting $\varepsilon_r = 0$ with the linearised coefficients of Equation 6.3.1 and $C_{z,0} = C_{m,0} = 0$ gives the trim angle of attack,

$$\alpha_{trim} = -\frac{M_{\theta_t}}{I\,\partial\varepsilon_r/\partial\alpha} = \frac{-(l_g - l_t)T\sin\theta_t}{qS\big[(l_g - l_p)C_{z,\alpha} - c\,C_{m,\alpha}\big]}. \tag{6.2.5}$$

A stable airframe has a negative denominator, so α_{trim} carries the sign of M_{θ_t}: the rocket flies at a small constant angle off the relative wind, whose normal force curves the trajectory, rather than tumbling. $\alpha_{trim} \propto T/q$ grows without bound just after ignition and in any high-altitude burn, and as the static margin tends to zero — the physical reason launch rails exist.

The radial offsets r_g, r_t enter Equation 6.2.1 on the same footing as θ_t, adding the constant offset $(r_g - r_t)T\cos\theta_t - r_g\,qSC_x$; the worked cases set them to zero to isolate θ_t. A commanded, time-varying $\theta_t(t)$ remains future work.

6.3 Static stability

Static stability asks whether a disturbance in the angle of attack α produces a restoring moment. Take the no-thrust aligned case, Equation 6.2.2 with $r_g = 0$, and linearise the coefficients,

$$C_z = C_{z,0} + \alpha\,C_{z,\alpha}, \qquad C_m = C_{m,0} + \alpha\,C_{m,\alpha}, \tag{6.3.1}$$

with slopes $C_{z,\alpha} = \partial C_z/\partial\alpha$, $C_{m,\alpha} = \partial C_m/\partial\alpha$ and offsets $C_{z,0}$, $C_{m,0}$. This is an analysis device applied to whichever coefficients are in force; the Linearized model (subsubsection 3.4.4) is its form in α, M and Re. Stability requires $\partial\varepsilon_r/\partial\alpha < 0$: a disturbance must produce an angular acceleration that reduces it. The derivative is

$$\frac{\partial\varepsilon_r}{\partial\alpha} = \frac{qS}{I}\Big[(l_g - l_p)C_{z,\alpha} - c\,C_{m,\alpha}\Big], \tag{6.3.2}$$

negative for $C_{z,\alpha} > 0$ (the normal case) when

$$l_g - l_p < c\,\frac{C_{m,\alpha}}{C_{z,\alpha}}. \tag{6.3.3}$$

Both lengths are measured from the nose, so $l_g - l_p < 0$ exactly when p sits aft of g — the familiar "centre of pressure behind centre of gravity" rule, which is what Equation 6.3.3 reduces to when p is the centre of pressure itself, $C_{m,\alpha} = 0$. In this report's sign of $\vec{A}_m$, a positive $C_{m,\alpha}$ is stabilising, the opposite of the textbook $C_{m,\alpha} < 0$.

Static margin, in calibers. p is a moment reference station, not necessarily the centre of pressure (subsection 2.3): the CP is the derived station at which the moment slope about it vanishes,

$$l_{cp} = l_p - c\,\frac{C_{m,\alpha}}{C_{z,\alpha}}, \tag{6.3.4}$$

and the rocketry convention divides the CP-to-CoG arm by the reference length, flipping the sign so a stable rocket gets a positive number:

$$\mathrm{SM} = \frac{l_{cp} - l_g}{c} = \frac{l_p - l_g}{c} - \frac{C_{m,\alpha}}{C_{z,\alpha}}. \tag{6.3.5}$$

$\mathrm{SM} > 0$ is then equivalent to Equation 6.3.3; the two coincide, $l_{cp} = l_p$, exactly when $C_{m,\alpha} = 0$ about p (the degenerate case where p already is the CP).

6.4 Flight-path angle and the point-mass gravity turn

Everything above tracks the *nose* direction θ_r. The trajectory itself is set by the *velocity* direction, and the two differ by the angle of attack. Define the flight-path angle θ_v, the angle of $\vec{v}_r$ above the horizon (conventionally written γ; here θ_v, since γ_r is the roll angle of subsection 4.4), by the same construction as Equation 2.1.1:

$$\hat{x}_v = \cos\theta_v\,\hat{x}_0 - \sin\theta_v\,\hat{z}_0, \qquad \hat{z}_v = \sin\theta_v\,\hat{x}_0 + \cos\theta_v\,\hat{z}_0, \qquad \vec{v}_r = V\hat{x}_v, \tag{6.4.1}$$

so that $v_x = V\cos\theta_v$, $v_z = -V\sin\theta_v$ and $\theta_v = \operatorname{atan2}(-v_z, v_x)$: positive climbing, since $\hat{z}_0$ points down. In the pitch plane and in still air the three angles close exactly,

$$\alpha = \theta_r - \theta_v, \tag{6.4.2}$$

which is the simulator's $\alpha = \operatorname{atan2}(v_{rel,z}, v_{rel,x})$ of subsubsection 3.4.1 written in ground-frame terms. With wind, θ_v in Equation 6.4.2 is the air-relative flight-path angle.

The point-mass limit. Strip the airframe of its rotational degree of freedom: no aerodynamic normal force, thrust along the velocity vector ($\theta_t = 0$ and $\alpha = 0$, so $\hat{x}_v = \hat{x}_r$), a flat non-rotating Earth ($\hat{z}_g = \hat{z}_0$, $\varphi = 0$), still air (so the path speed is the airspeed, $V = v_{rel}$) and drag D antiparallel to $\vec{v}_r$. Then Equation 2.4.1 reduces to $m\vec{a}_r = mg\hat{z}_0 + (T - D)\hat{x}_v$, while differentiating Equation 6.4.1 gives

$$\vec{a}_r = \dot{V}\hat{x}_v - V\dot{\theta}_v\,\hat{z}_v. \tag{6.4.3}$$

Projecting on $\hat{z}_v$, where $\hat{z}_0 \cdot \hat{z}_v = \cos\theta_v$ and weight is the only contributor, and on $\hat{x}_v$:

$$\dot{\theta}_v = -\frac{g}{V}\cos\theta_v, \tag{6.4.4}$$

$$m\dot{V} = T - D - mg\sin\theta_v. \tag{6.4.5}$$

Equation 6.4.4 is the *gravity turn*: gravity alone bends the velocity vector, at a rate g/V modulated by $\cos\theta_v$. It vanishes when the flight is vertical ($\theta_v = \pm 90°$), where gravity is purely decelerating and turns nothing, and is largest when level, $\dot{\theta}_v = -g/V$. A faster rocket turns less per unit time — the reason a high thrust-to-weight launch holds its initial attitude. Holding V fixed, Equation 6.4.4 integrates to

$$\tan\left(\frac{\pi}{4} + \frac{\theta_v}{2}\right) = \tan\left(\frac{\pi}{4} + \frac{\theta_{v,0}}{2}\right) e^{-gt/V}, \tag{6.4.6}$$

so $\theta_v \to -90°$ with time constant V/g.

This is a *cruder* model than Equation 2.4.1–Equation 2.4.3, not a refinement of them: it carries no moment, no inertia and no angle of attack, and the simulator never integrates it. Its uses are the two below — a closed-form estimate to check the integrator against, and the justification of the short-period approximation that follows.

6.4.1 Dynamic stability

The static criterion does not say what the resulting oscillation does. That needs the **pitch-damping derivative** $C_{m,q}$, the sensitivity of C_m to the non-dimensional pitch rate $\hat{\omega} = \omega_r c/(2v_{rel})$:

$$C_m = C_{m,0} + \alpha\,C_{m,\alpha} + \hat{\omega}\,C_{m,q}. \tag{6.4.7}$$

It comes from the body's own rotation. A point a distance s_i aft of the CoG moves relative to the CoG with velocity $\omega_r\hat{y}_r \times \vec{s}_i$ and sees a local angle of attack shifted by $\Delta\alpha_i \approx +\omega_r s_i/v_{rel}$. Fins well aft see the largest shift, and the resulting normal-force change against its arm gives a moment proportional to ω_r, present even at $\alpha = 0$.

With the sign of $\vec{A}_m$ used here, a positive C_m pitches the nose *down*. The simulator's code flags this non-standard sign by renaming the internal quantity to `C_m_code`, keeping the plain name `C_m` reserved at the code's I/O boundary (config files, CSV aero-table headers) for the standard convention when a user declares `cm_convention: standard`. A statically stable, aft-finned airframe therefore has $C_{m,q} > 0$: a positive (nose-up) pitch rate produces an additional positive (nose-down) moment, opposing the rotation.

Linearising Equation 6.2.3 with this term (and $C_{z,0} = C_{m,0} = 0$),

$$I\varepsilon_r = qS(l_g - l_p)C_{z,\alpha}\,\alpha - qSc\,C_{m,\alpha}\,\alpha - \frac{qSc^2}{2v_{rel}}C_{m,q}\,\omega_r, \tag{6.4.8}$$

which separates into a stiffness term and a damping term,

$$M_\alpha \equiv \frac{\partial \varepsilon_r}{\partial \alpha} = \frac{qS}{I}\big[(l_g - l_p)C_{z,\alpha} - c\,C_{m,\alpha}\big], \qquad M_q \equiv \frac{qSc^2}{2v_{rel}I}C_{m,q}. \tag{6.4.9}$$

M_α is negative for a statically stable airframe and M_q is positive for $C_{m,q} > 0$. Eliminating θ_r in favour of α through Equation 6.4.2 ($\omega_r = \dot\alpha + \dot\theta_v$, $\varepsilon_r = \ddot\alpha + \ddot\theta_v$) turns $\varepsilon_r = M_\alpha\alpha - M_q\omega_r$ into an exact statement,

$$\ddot\alpha + M_q\dot\alpha - M_\alpha\alpha = -\left(\ddot\theta_v + M_q\dot\theta_v\right), \tag{6.4.10}$$

whose right-hand side is the entire content of the short-period approximation: dropping it gives

$$\frac{\mathrm{d}^2\alpha}{\mathrm{d}t^2} + M_q\frac{\mathrm{d}\alpha}{\mathrm{d}t} - M_\alpha\alpha = 0, \tag{6.4.11}$$

with natural frequency and damping ratio

$$\omega_n = \sqrt{-M_\alpha}, \qquad \zeta = \frac{M_q}{2\omega_n} = \frac{M_q}{2\sqrt{-M_\alpha}}, \tag{6.4.12}$$

valid only for $M_\alpha < 0$; otherwise the disturbance grows rather than oscillates.

Equation 6.4.4 makes the dropped term quantitative instead of merely "slow". With $|\dot\theta_v| \le g/V$ and $\ddot\theta_v = O\Big((g/V)^2 + g\dot V/V^2\Big)$, the dominant neglected term is $M_q\dot\theta_v = 2\zeta\omega_n\dot\theta_v$. Measuring it against the retained stiffness $M_\alpha\alpha \sim \omega_n^2\hat\alpha$ for an oscillation of amplitude $\hat\alpha$, and writing

$$\Lambda \equiv \frac{g/V}{\omega_n} = \frac{g}{V\sqrt{-M_\alpha}}, \tag{6.4.13}$$

the approximation holds when $2\zeta\Lambda \ll \hat\alpha$ and $\Lambda^2 \ll \hat\alpha$ — in short, $\Lambda \ll 1$: the flight-path angle must drift through much less than the oscillation amplitude in one short period, $\Delta\theta_v \approx 2\pi\Lambda \ll \hat\alpha$. Since $\Lambda \propto 1/V$, the approximation is at its weakest at low airspeed — on and just off the rail, and near the apogee of a lofted flight — and strongest through the high-q part of the boost. Near-vertical flight relaxes it independently of speed, because $\cos\theta_v \to 0$ suppresses $\dot\theta_v$ directly.

The distributed-force argument above gives an estimate. A component i with normal-force slope $C_{z,\alpha,i}$ (referred to S) at a distance s_i from the CoG adds a moment proportional to $\omega_r s_i^2/v_{rel}$, and equating the sum with the $C_{m,q}$ term gives

$$C_{m,q} \approx \frac{2}{c^2}\sum_i C_{z,\alpha,i}\,s_i^2, \tag{6.4.14}$$

positive in this report's sign convention. OpenRocket uses a different estimate, from the cross-flow drag of the rotating body and fins [5]. (research: a value of $C_{m,q}$ for the project's airframe: the component slopes $C_{z,\alpha,i}$, or a measured value (forced-oscillation tests or unsteady CFD))

7 Current assumptions

The simplifications the derivation and the simulator rely on. Each is expected to be relaxed later, unless stated otherwise.

- **Planar flight, no crosswind.** Motion is confined to the vertical plane fixed by the launch azimuth χ; the out-of-plane wind component is computed and discarded. Wind is horizontal only: updrafts and downdrafts are not modelled. The yaw rate is $\omega_z \equiv 0$, which is what makes the roll freedom of subsection 4.4 decouple exactly rather than approximately.
- **Gravity direction.** Parallel by default, $\hat{z}_g \equiv \hat{z}_0$. The radial and ellipsoidal direction models are available; once one is enabled the ground is curved and the true altitude h_{true} (Equation 3.1.11) replaces $h = -z$, with impact at $h_{true} = 0$.
- **Earth rotation, in part.** The centrifugal effect on the magnitude of g and the in-plane Coriolis term are included. The out-of-plane Coriolis component and the residual centrifugal term are dropped, and $\vec{\Omega}$ is frozen at the launch site's latitude and heading.
- **Moment arms.** Mass, centre of gravity and inertia vary with the motor model. The centre of pressure l_p responds to the flight state only under the table aerodynamic model; the thrust point l_t is fixed.
- **Thrust axis.** The thrust angle θ_t and the radial offsets r_g, r_t are constant inputs, zero by default. A commanded gimbal is not modelled.
- **Rail, recovery, air brake.** The rail has no friction and no tip-off. The canopy opens instantly and the attitude is frozen under it. The air brake affects the axial force only, through a memoryless law; a law with memory is expressible in the sampled mode but not written.
- **Turbulence.** The Dryden spectrum is driven at a fixed reference airspeed, not the rocket's own, and has no vertical component. It is off by default.
- **Point-mass gravity turn.** The flight-path-angle rate of Equation 6.4.4 (subsection 6.4) is an analysis-only model: it strips the airframe of its rotational degree of freedom entirely (no aerodynamic normal force, thrust aligned with the velocity vector, a flat non-rotating Earth, still air) and is never integrated by the simulator, which always carries the full rotational dynamics of Equation 2.4.3.
- **Roll.** The roll freedom of subsection 4.4 is exact in the pitch plane, but the yaw torques that roll and the wheel produce — Equation 4.4.1's $(I_x - I)\omega_z\omega_x$ and Equation 4.4.2's $\omega_r h_w$ — cannot be represented at all and are dropped with ω_z. The aerodynamic coefficients do not depend on the roll angle, so fin–body interference at incidence, roll lock-in and the Magnus side force are absent; the wheel is balanced and centred on the axis, so spinning it moves neither the centre of gravity nor the pitch inertia. Off by default.
- **Three-dimensional motion.** section 5 lifts the two items above — planar flight and the planar roll freedom — but it is a formulation of the rotational dynamics, not yet a flyable mode, and it carries assumptions of its own. The inertia tensor (Equation 5.3.2) is *constant*: the 3D formulation is written for a rigid body of fixed mass distribution, so the variable-mass inertia $I(t)$ of Equation 3.3.7 and the term $\dot{\mathbf{I}}\vec{\omega}$ it would add are both absent. The moment vector (Equation 5.6.1) supplies the pitching component only: there is no yaw moment from sideslip and no roll moment from fin cant, so a 3D run with a symmetric tensor reproduces the planar pitch history and adds nothing. A reaction wheel, if present, is taken as locked (its inertia inside I_{xx}, its momentum outside $\vec{H}$). The rail constraint, the canopy phase and the flight-phase events are all written against the planar state (Equation 3.9.2) and have no 3D counterpart, so a 3D scenario is integrated over a fixed time span rather than flown to the ground. Off by default; the planar model remains the one every result in this document is produced with.

8 Backlog and future work

What is deliberately left undone, and what has to exist before each item can be written down.

- **Commanded gimbal control.** A time-varying $\theta_t(t)$ driven by a control law; a 3D item, since steering needs an out-of-plane degree of freedom. *Waiting on:* a guidance objective, actuator limits, a sensor model, and the 3D model. The air brake is a different loop, implemented in 2D with a proportional law; a law with integral or derivative memory on the zero-order-hold mode (signal, gains, saturation, anti-windup) is unwritten, as are brake retraction and a rate limit.
- **Wing-body aerodynamics.** Replace the constant, linearized and table coefficient models with a Barrowman-style build-up: each of nose, body and fins contributes a normal-force slope $C_{z,\alpha,i}$ at a location X_i, and the centre of pressure is the weighted average $X_{cp} = \sum_i C_{z,\alpha,i} X_i / \sum_i C_{z,\alpha,i}$, so that $l_p - l_g$ is computed rather than read from a table. A Mach correction is needed beyond low subsonic speed. *Waiting on:* an actual airframe — nose shape, body diameter and length, fin planform.
- **Dynamic stability, quantitatively.** The damped short-period framework exists; the value of $C_{m,q}$ does not. *Waiting on:* a measured value (forced-oscillation tests or unsteady CFD) or a distributed-force derivation from the airframe.
- **Propellant grain geometry** behind $I_{m,g}(t)$ and $l_{g,m}(t)$: choose a grain family and its dimensions, from which $A_b(w)$, $l_{g,m}(t)$ and $I_{m,g}(t)$ follow by integration.
- **Component-based mass properties** of the dry rocket, built from nose cone, tubes, fins and internal parts with their densities. *Waiting on:* the airframe geometry and a materials table.
- **Vertical wind.** Terrain-slope orographic lift or a convective parameterisation; each needs a data source the project does not have. On hold pending research.
- **CFD pipeline** for C_x, C_z, C_m: the run matrix or wind-tunnel campaign that populates the aerodynamic table for a specific airframe.
- **Implementation appendix** — the simulator's algorithms (integrator stepping, event location, table interpolation, ...) documented from the code.
- **Out-of-plane motion**: crosswind weathercocking and the lateral Coriolis deflection, both lifted by a full 6DOF model.
- **State-coupled turbulence.** A Dryden filter driven by the true airspeed, carrying its own state on the sampled mode. *Waiting on:* its discretisation, and whether per-tick noise makes results depend on the sample rate.
- **Rail–rocket dynamics, in detail.** Derive the reaction (normal) forces at the rail buttons and the friction they cause, and the tip-off transient at rail exit. *Waiting on:* button positions, clearances and a friction coefficient.
- **Rocket and parachute dynamics, in detail.** A coupled two-body model: pendulum motion (wobble) of the rocket under the canopy, riser elasticity, and the canopy opening transient with its inflation shock. *Waiting on:* canopy and riser data, and the attitude degree of freedom under canopy.
- **Roll dynamics with a reaction wheel (2D)** — derived in subsection 4.4; what remains is data, not derivation. *Waiting on:* I_x of the airframe and the chosen wheel's I_w, τ_{max}, $\omega_{w,max}$; the aerodynamic roll coefficients C_l and $C_{l,p}$, which need a chosen fin cant and either a Barrowman-style build-up or CFD; and the motor's own roll torque (subsection 3.7), for which no prediction method is known. A roll law with memory — an integral term, a torque rate limit, or a measured rather than true ω_x — additionally needs a roll channel on the sampled mode (subsection 3.10).
- **Model extensions for 3D.** The attitude representation, the inertia tensor and the rotational equations of motion are now derived in section 5; what each remaining model needs beyond the pitch plane is:
  - gravity: the full vector field, not only the in-plane direction $\hat{z}_g$;
  - thrust: the $\hat{y}_t$ component of the line of thrust and the motor roll torque;
  - mass: the tensor of Equation 5.3.2 is derived but constant — what remains is its time dependence as the grain burns (the $\dot{\mathbf{I}}\vec{\omega}$ term of subsection 5.3), the off-axis centre of gravity, and measured values for I_{xx} and the three products of inertia, which no source supplies today;
  - moment arms: the radial offsets r_g, r_t of Equation 2.3.2 as 3D vectors (subsection 5.6);

- aerodynamics: side-force, yaw- and roll-moment coefficients as functions of sideslip β as well as α, which is what fills in M_x and M_z of Equation 5.6.1;
- wind and turbulence: the lateral mean-wind component and the Dryden v and w components;
- Earth rotation: the full Coriolis vector, with $\vec{\Omega}$ resolved along the trajectory;
- rail, recovery and air brake: the rail constraint with lateral tip-off, canopy drag and riser pendulum in 3D, and a brake that also yaws or rolls the airframe;
- integration: the flight-phase machinery — ground, apogee and burnout events, the rail hold and the canopy hold — is written against the planar state (Equation 3.9.2) and has to be generalised before any 3D scenario can be flown to the ground (subsection 5.7). This is the largest single item of the group.

A Notation

The table maps each symbol to the identifier the simulator uses. "Not stored": the code keeps only an arm relative to the centre of gravity; "not implemented": the quantity appears in the report but not in the simulator.

Symbol	Code identifier	Meaning	Units
Geometry and frames			
A	—	nose tip, origin of all lengths	—
g	—	centre of gravity	—
t	—	thrust application point	—
p	—	aerodynamic reference point (centre of pressure)	—
l_g	`mass.cog_position(m_p, p)`; local `l_B` in `apps/sim/engine/dynamics.py` (mass model only)	axial distance $A \to g$	m
r_g	`p.lengths.u_B`	radial offset of g	m
l_t	not stored; $l_t - l_g =$ `p.lengths.l1 + p.lengths.l2`	axial distance $A \to t$	m
r_t	`p.lengths.u_C`	radial offset of t	m
l_p	not stored; $l_p - l_g =$ `l1` (row below)	axial distance $A \to p$	m
$r_p \equiv 0$	—	no radial offset of p, by definition	m
$l_p - l_g$	`p.lengths.l1`; `aero.coefficients(...).l1` (varies under the table model)	arm $g \to p$, positive with p aft	m
$l_t - l_p$	`p.lengths.l2`	arm $p \to t$	m
$\vec{gt}$, $\vec{gp}$	`aB`, `aA` in `apps/sim/engine/dynamics.py`	position of t, p relative to g	m
θ_t	$-$`np.radians(p.lengths.misalign_deg)`	thrust-line angle from $\hat{x}_r$	rad (code: deg)
Mass			
m, I	`p.weight.m`, `p.weight.I`, or `mass.moment_of_inertia`	mass; pitch moment of inertia about g	kg, kg m^2
m_r	`airframe.m` (`apps/sim/schema.py`); `p.weight.m_d` also includes the casing	dry-rocket mass (constant)	kg
$m_m(t)$	`mass.propellant_mass(m, p)` (`y[M] − m_d`) is the propellant only; m_m adds the casing m_f	motor mass (casing and propellant), burned down to m_f	kg
m_0, m_f	`motor.m_total`; $m_f =$ `m_total - m_prop`	motor mass at ignition, at burnout	kg
$\dot{m}_m$	`dydt[M]` in `apps/sim/engine/dynamics.py` (the state m follows it)	motor mass flow, negative	kg/s
Δm	`motor.m_prop`	burned mass $m_0 - m_f$	kg
ISP	`p.weight.I_sp`; `motor.I_sp`	specific impulse	s
g_0	`p.g0`	standard gravity	m/s^2
I_r, $I_{r,g}$	`p.weight.I_d` (dry assembly about its own centroid)	dry-rocket inertia about g; about its own centre of gravity	kg m^2
$I_m(t)$, $I_{m,g}(t)$	`p.weight.I_p_spec`$\cdot m_p$ (propellant only)	motor inertia about g; about its own centre of gravity	kg m^2
I_0, I_f	`motor.I_0`, `motor.I_f`	$I_{m,g}$ at ignition, at burnout	kg m^2
$l_{g,r}$, $r_{g,r}$	`p.weight.l_d`; radial: not stored (constant `u_B`)	dry-rocket centre of gravity (axial, radial)	m
$l_{g,m}(t)$, $r_{g,m}(t)$	`p.weight.l_p` (fixed); radial: not stored	motor centre of gravity (axial, radial)	m
l_0, l_f	`motor.l_cog_0`, `motor.l_cog_f`	$l_{g,m}$ at ignition, at burnout	m
Forces and aerodynamics			
S	`p.aero.S`	aerodynamic reference area	m^2
c	`p.aero.d_ref`	reference length of the pitching moment	m
C_x, C_z, C_m	`p.aero.C_x`, `C_z`, `C_m` (constant model); fields of `aero.AeroCoeffs`	axial-force, normal-force, pitching-moment coefficients	—

Symbol	Code identifier	Meaning	Units
qSC_x, qSC_z	`forces.aero_force_body` returns $(-A_x, -A_z)$	aerodynamic force components	N
$qScC_m$	`forces.aero_moment_y` returns $-A_m$	aerodynamic pitching moment	N m
S_k, c_k, $C_{x,k}$, $C_{z,k}$, $C_{m,k}$, pp_k	not implemented	aero component k: reference area and length, coefficients, reference point	m^2, m, —, m
$C_{x,0}$, C_{x,α^2}	not implemented	axial coefficient at $\alpha = 0$ and its α^2 coefficient	—, $1/rad^2$
$\partial C/\partial\alpha$, $\partial C/\partial M$, $\partial C/\partial Re$	only the α slope is in the code	linearized-model slopes	1/rad, —, —
$C_{z,\alpha}$, $C_{m,\alpha}$, $C_{m,q}$, $C_{z,0}$, $C_{m,0}$	aero coefficient inputs (`c_mq` column of the aero table, ...)	coefficient slopes and offsets	1/rad or —
Thrust			
T, $\vec{T}$	`thrust_curve.thrust`; `forces.thrust_body`	thrust magnitude; vector in $\mathcal{F}_r$	N
$T_0(t)$	`thrust_curve` (`T_ref`)	reference (test-stand) thrust	N
p_0	`p.thrust.p_ref`	ambient pressure during the static test	Pa
A_e	`p.thrust.A_e`	nozzle exit area	m^2
$\hat{z}_t$	—	thrust-frame normal (Equation 3.2.4)	—
$\vec{T}_s$, κ, $\xi(t)$, τ_c	not implemented	random side force of the thrust, its intensity (fraction of T), unit random process, correlation time	N, —, —, s
ρ_{pr}, a_b, n	not implemented	propellant density, burn-rate coefficient and exponent ($r = a_b p_c^n$)	kg/m^3, $m/s{\cdot}Pa^{-n}$, —
l_e	not stored; $\approx l_t - l_g$	arm from g to the nozzle exit (jet damping)	m
Atmosphere and wind			
p_{atm}	`air_state(...).p`	ambient pressure	Pa
q	`flow.q`	dynamic pressure	Pa
v_{rel}, $\vec{v}_{rel}$	`flow.V`	airspeed; relative-wind vector	m/s
$v_{rel,x}$, $v_{rel,z}$	—	relative-velocity components in $\mathcal{F}_r$	m/s
α	`flow.alpha`	angle of attack	rad
θ_v	not implemented (derivable: `atan2(-vz, vx)`, or `theta - alpha` in still air)	flight-path angle, velocity vector above the horizon	rad
$\hat{x}_v$, $\hat{z}_v$	—	velocity-axis unit vectors (Equation 6.4.1)	—
V	`flow.V` (air-relative v_{rel}); $\lvert\vec{v}_r\rvert$ inertially	point-mass path speed; $V = v_{rel}$ in still air	m/s
w_{up}	not implemented	updraft speed ($w_z = -w_{up}$)	m/s
v_{ref}	turbulence setting	fixed reference airspeed of the Dryden spectrum	m/s
States and integration			
$\vec{y} = (x, z, v_x, v_z, \theta_r, \omega_r, m)$	`y[X,Z,VX,VZ,THETA,OMEGA,M]`	integrated state vector	m, m/s, rad, rad/s, kg
θ_r	`y[THETA]`	pitch angle	rad
ω_r	`y[OMEGA]`	pitch rate	rad/s
ε_r	`M_y / I` ($d\omega/dt$ in `apps/sim/engine/dynamics.py`)	angular acceleration	rad/s^2
$\vec{x}_r$	`y[X]`, `y[Z]`	position in $\mathcal{F}_0$	m
$\vec{v}_r$	`y[VX]`, `y[VZ]`	velocity in $\mathcal{F}_0$	m/s
$\vec{v}_g$	`y[VX]`, `y[VZ]`	CoG velocity (the same vector as $\vec{v}_r$)	m/s
v_x, v_z	`y[VX]`, `y[VZ]`	velocity components in $\mathcal{F}_0$	m/s
$\vec{a}_r$	`a_i` in `apps/sim/engine/dynamics.py`	acceleration in $\mathcal{F}_0$	m/s^2
m_{launch}	`s0.m`; `p.weight.m`	liftoff mass $m(0)$	kg
h	`p.solver.dt`	fixed RK4 step	s

Symbol	Code identifier	Meaning	Units
θ_{rail}, L_{rail}, s_{rail}, $\hat{e}_{rail}$	`p.site.rail_angle_deg`, `p.site.L_rail`, `rail_distance`, `_e_rail`	rail elevation, length, distance travelled, direction	rad, m, m, —
Recovery and air brake			
S_{ch}, $C_{D,ch}$, $\vec{R}_{ch}$, v_{term}	`p.recovery.S`, `p.recovery.Cd`, `forces.parachute_drag_inertial`	canopy area, drag coefficient, recovery (canopy drag) force, terminal speed	m^2, —, N, m/s
$(C_D S)_{sys}$	not implemented	combined drag area of rocket and canopy (lumped descent)	m^2
f_b	`control.airbrake_deploy_fraction`	air-brake deployment fraction	—
S_b, $C_{D,b}$	`p.control.airbrake_S_max`, `airbrake_Cd`	air-brake area at full deployment, drag coefficient	m^2, —
F_b	`forces.aero_force_body` (`A_brake`)	air-brake axial force	N
c_b, $C_{x,b}$, $C_{z,b}$, $C_{m,b}$, pp_b	not implemented	air-brake as an aerodynamic component: reference length, coefficients, reference point	m, —, m
τ_s, f_s	sampling setting	sample period and rate of the sampled mode	s, Hz
Analysis			
$\hat{\omega} = \omega_r c/(2v_{rel})$	—	non-dimensional pitch rate	—
$M_{\theta_t} = (l_g - l_t)\, T \sin\theta_t$	—	misalignment disturbance torque	N m
s_i	—	position of a body point relative to the CoG ($C_{m,q}$ derivation)	m
M_α, M_q, ω_n, ζ	—	short-period parameters	$1/s^2$, 1/s, rad/s, —
$\Lambda = (g/V)/\omega_n$	—	flight-path drift rate against the short-period frequency	—

Some letters carry two meanings, told apart by context: g is the centre of gravity, the gravitational acceleration, and as a subscript both the point (l_g, r_g) and the gravity direction ($\hat{z}_g$); t as a point differs from time t; h is the RK4 step in subsection 3.9 but the altitude elsewhere. l_p is only the aerodynamic reference point; in the code `p.weight.l_p` is the propellant centroid (the motor's $l_{g,m}$). θ now carries three subscripted meanings: body pitch (θ_r), thrust-line angle (θ_t) and flight-path angle (θ_v, subsection 6.4). The conventional symbol for the flight-path angle, γ, is deliberately not used here, since γ_r already names the roll angle (subsection 4.4).

B Model inventory

Every symbol on the right-hand sides of Equation 2.4.2 and Equation 2.4.4 is either a state or is supplied by a model:

- m, I — mass and moment of inertia, from the mass model (subsection 3.3);
- $g\hat{z}_g$ — gravity, magnitude and direction (subsection 3.1);
- T — thrust curve plus pressure correction (subsection 3.2);
- θ_t — thrust direction: can be controlled, for now a constant, ideally zero (subsection 3.2);
- q — from the atmosphere model, the state and the wind model, through ρ and v_{rel} (subsection 3.5, subsection 3.6; subsubsection 3.4.1 in subsection 3.4);
- S, c — reference area and reference length (subsection 3.4);
- C_x, C_z, C_m — aerodynamic coefficients (subsection 3.4);
- l_t, r_t, l_p, l_g, r_g — lengths (subsection 2.3); $l_g(t)$ moves with the mass model, and the position of p comes with the coefficients;
- states — θ_r, ω_r, $\vec{x}_r$, $\vec{v}_r$ are integrated, the equations deliver ε_r and $\vec{a}_r$ (subsection 3.9);
- and, outside free flight, the parachute $\vec{R}_{ch}$ (subsection 4.2), the air brake (subsection 4.3) and the launch rail (subsection 4.1), all in section 4.

C Conventions

Signs and axes in one place; the rest of the report follows them.

- **Frames.** $\hat{x}$ points forward (downrange for $\mathcal{F}_0$, along the nose for $\mathcal{F}_r$), $\hat{z}$ points down (towards the belly for $\mathcal{F}_r$), and $\hat{y}$ completes the right-handed triad, $\hat{x} \times \hat{y} = \hat{z}$; $\hat{y}$ is out of the pitch plane, to the right of the downrange direction.
- **Pitch.** θ_r is measured from $\hat{x}_0$, positive nose-up; ω_r and ε_r share its sign (rotation about $+\hat{y}_r$).
- **Roll.** γ_r is measured about $+\hat{x}_r$ by the right-hand rule, so a positive γ_r turns $\hat{y}_r$ towards $\hat{z}_r$; ω_x and ε_x share its sign. The wheel torque τ_w is the torque on the *wheel* about $+\hat{x}_r$, so the airframe receives $-\tau_w$ (Equation 4.4.4), and the wheel rate ω_w is absolute, not relative to the airframe. The roll coefficients follow the same code sign as $C_{m,q}$ — positive $C_{l,p}$ damps — and are *not* affected by the C_m convention switch, which governs the pitching moment only.
- **Inertia tensor.** The three products of inertia are the *integrals* $I_{xy} = \int xy\,\mathrm{d}m$, $I_{xz} = \int xz\,\mathrm{d}m$, $I_{yz} = \int yz\,\mathrm{d}m$ (Equation 5.3.1), so the inertia matrix carries minus signs off the diagonal (Equation 5.3.2). Much of the physics literature instead names the matrix entries themselves, i.e. the negatives of these; off-diagonal numbers taken from such a source must be sign-flipped before use. All six components are referred to the centre of gravity g and to the body axes $\hat{x}_r, \hat{y}_r, \hat{z}_r$, which are *not* assumed to be principal axes. In the planar notation I_{yy} is the pitch inertia I (subsubsection 3.3.3) and I_{xx} is the axial inertia I_x (subsection 4.4).
- **Lengths.** l is measured from the nose tip A positive aft, so a point sits at $-l\,\hat{x}_r$; the radial offset r is along $-\hat{z}_r$, so it sits at $-r\,\hat{z}_r$.
- **Moments** are taken about the centre of gravity g, positive about $+\hat{y}_r$ (nose-up is positive θ_r).
- **Thrust angle.** $\hat{x}_t = \cos\theta_t\,\hat{x}_r - \sin\theta_t\,\hat{z}_r$ (Equation 3.2.4): positive θ_t tilts the thrust line the way positive θ_r tilts the nose, and pitches the nose down through a nozzle aft of g.
- **Aerodynamics.** $\vec{A}_f = -qS\,(C_x\hat{x}_r + C_z\hat{z}_r)$ and $\vec{A}_m = -qSc\,C_m\hat{y}_r$, so positive C_x and C_z oppose motion along $+\hat{x}_r$ and $+\hat{z}_r$, and positive C_m pitches the nose *down*. This is the opposite of the usual aerospace sign: here static stability needs $C_{m,\alpha} > 0$ and a damping $C_{m,q} > 0$, as in the simulator, which applies $-qSc\,C_m$ to the pitch moment.
- **Units.** SI throughout; angles are radians internally and degrees in the configuration files.

D Constants

WGS84 ellipsoid constants used by the ellipsoidal gravity model, the standard gravity and the ISA layer table.

Symbol	Value	Units	Meaning
a_e	6378137	m	equatorial radius
f	1/298.257223563	—	flattening
e^2	0.00669437999013	—	first eccentricity squared
m	0.00344978650684	—	$\Omega^2 a_e^2 b_e / GM$ (rotation-to-gravity ratio)
γ_e	9.7803253359	m/s^2	normal gravity at the equator
k	0.00193185265241	—	Somigliana constant
g_0	9.80665	m/s^2	standard gravity

ISA layers of the atmosphere model (subsection 3.5), with base geopotential altitude H_b, lapse rate L, base temperature and pressure T_b, p_b:

Layer	H_b [m]	L [K/m]	T_b [K]	p_b [Pa]
troposphere	0	−0.0065	288.15	101325
tropopause	11000	0	216.65	22632.06
stratosphere	20000	+0.0010	216.65	5474.889
stratosphere	32000	+0.0028	228.65	868.0187
stratopause	47000	0	270.65	110.9063
mesosphere	51000	−0.0028	270.65	66.93887
mesosphere	71000	−0.0020	214.65	3.956420

Table 2: ISA layers.

References

[1] G. P. Sutton, O. Biblarz, *Rocket Propulsion Elements*, 9th ed., Wiley, ch. 2.

[2] OpenRocket developers' mailing list, on the motor mass model, 2012, https://sourceforge.net/p/openrocket/mailman/message/29671720/.

[3] HPR Sim physics documentation, motor model (after RocketPy `SolidMotor`), https://hpr.fusionspace.co/physics/motor.html.

[4] OpenRocket user guide, *Overrides and surface finish*, https://openrocket.readthedocs.io/en/latest/user_guide/overrides_and_surface_finish.html.

[5] OpenRocket technical documentation, https://openrocket.sourceforge.net/techdoc.pdf.

[6] Kowaleczko et al., *J. Theor. Appl. Mech.* 54(3), 2016, https://doi.org/10.15632/jtam-pl.54.3.893.

[7] Pitch jet damping of rocket-assisted projectiles, DTIC ADA089980, https://apps.dtic.mil/sti/pdfs/ADA089980.pdf.

[8] Knauber, thrust misalignments of fixed-nozzle solid rocket motors, AIAA Paper 95-2873, https://doi.org/10.2514/6.1995-2873.

[9] R. N. Knauber, Roll torques produced by fixed-nozzle solid rocket motors, *J. Spacecraft Rockets* 33(6), 1996, https://doi.org/10.2514/3.26839.

[10] Flying qualities of piloted airplanes, MIL-F-8785C, 1980 (Dryden turbulence model; see also MIL-HDBK-1797). Forms used here after the MathWorks *Dryden Wind Turbulence Model (Continuous)* documentation, https://www.mathworks.com/help/aeroblks/drydenwindturbulencemodelcontinuous.html.

[11] S. Whitmore, Utah State University MAE 6530 lecture notes, launch vehicle design, recovery, https://mae-nas.eng.usu.edu/MAE_6530_Web/New_Course/launch_design/Section3.5.pdf.

[12] HPR Sim physics documentation, recovery, https://hpr.fusionspace.co/physics/recovery.html.

[13] Farha, air-brake CFD and trajectory study, M.S. thesis, West Virginia University, 2025, https://researchrepository.wvu.edu/cgi/viewcontent.cgi?article=13583&context=etd.

[14] T. W. Knacke, *Parachute Recovery Systems Design Manual*, NWC TP 6575.

[15] Small-UAS parachute wind-tunnel study, *J. Aerosp. Technol. Manag.*, https://www.scielo.br/j/jatm/a/rDFPNWbHLhSVrGTY4PhqtQB/.

[16] RASAero II user's manual, https://www.rasaero.com/dloads/RASAero%20II%20Users%20Manual.pdf.

[17] RASAero II release history, http://www.rasaero.com/dl_software_ii.htm.

[18] Missile DATCOM user's guide, http://mae-nas.eng.usu.edu/MAE_5900_Web/5900/USLI_2010/Flight_Mechanics/missile_datcom/Missile%20DATCOM%20Users%20Guide.pdf.

[19] Cobalt CFD dynamic derivatives, AIAA Paper 2016-1536, http://www.cobaltcfd.com/pdfs/2016/AIAA-2016-1536.pdf.

[20] N. K. Pavlis, S. A. Holmes, S. C. Kenyon, J. K. Factor, The development and evaluation of the Earth Gravitational Model 2008 (EGM2008), *J. Geophys. Res.* 117, B04406, 2012, https://doi.org/10.1029/2011jb008916.

[21] L. H. Jorgensen, Prediction of static aerodynamic characteristics for space-shuttle-like and other bodies at angles of attack from 0 to 180 deg, NASA TN D-6996, 1973, https://ntrs.nasa.gov/api/citations/19730006261/downloads/19730006261.pdf.

[22] J. C. van der Ha, R. Janssens, Jet damping and misalignment effects during solid-rocket-motor burn, *J. Spacecraft Rockets* 42(2), 2005, http://www.vanderha.com/.

[23] R. Nakka, Solid rocket motor design, https://www.nakka-rocketry.net/design1.html.

[24] openMotor, https://github.com/reilleya/openMotor/.

[25] J. R. Wertz (ed.), *Spacecraft Attitude Determination and Control*, Kluwer, app. D (quaternion algebra and kinematics).

[26] R. C. Hibbeler, *Engineering Mechanics: Dynamics*, Pearson, ch. 21 (three-dimensional kinetics of a rigid body; moments and products of inertia, Euler's equations of motion).

[27] B. L. Stevens, F. L. Lewis, E. N. Johnson, *Aircraft Control and Simulation*, 3rd ed., Wiley, ch. 1–2 (body-axis inertia tensor, quaternion attitude propagation).